

BASE DE DATOS 2

- **TÍTULO: TPI FOOD STORE - SISTEMA DE GESTIÓN DE PEDIDOS DE COMIDA**
- **INFORME TÉCNICO - PRIMERA ENTREGA PARCIAL (UNIDADES 1 A 3)**
- **INSTITUCIÓN: UTN - FACULTAD REGIONAL MENDOZA**
- **CARRERA: TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN**
- **MATERIA: BASE DE DATOS 2**
- **INTEGRANTES: MARIANO CHIRINO, FACUNDO QUIROGA, ANDRÉS FABRE.**
- **FECHA DE ENTREGA: SEPTIEMBRE DE 2026**

Índice

1. Resumen de la entrega y cobertura de los objetivos	3
2. Alcance, entorno y reproducción	3
2.1 Entorno y archivos del proyecto	3
2.2 Verificación de reproducibilidad	4
2.3 Volumen de datos de prueba	4
3. Marco teórico	5
4. Unidad 1 — Modelo de datos, integridad, transacciones y concurrencia	6
4.1 Modelo Entidad-Relación (objetivo 1)	6
4.2 Paso a modelo relacional (objetivo 2)	7
4.3 Normalización hasta 3FN/BCNF (objetivo 3)	7
4.4 DDL completo (objetivo 4)	8
4.5 Reglas de negocio (objetivo 7)	8
4.6 Transacciones y concurrencia (objetivo 8)	9
5. Unidad 2 — Consultas y optimización	12
5.1 Datos de prueba	12
5.2 Consultas: JOIN, agregación, subconsultas y funciones de ventana (objetivo 5)	12
5.3 Metodología de optimización	13
5.4 Resultados antes/después	14
5.5 Resumen de decisiones sobre índices	15
6. Unidad 3 — Índices, vistas y objetos programables	17
6.1 Índices del esquema final (objetivo 4)	17
6.2 Vistas (objetivo 6)	17
6.3 Función, triggers y procedimiento (objetivo 6)	18
6.4 Vista materializada (objetivo 6)	18
7. Borrado lógico (soft delete) y su impacto (objetivo 9)	19
7.1 Mecanismo	19
7.2 Impacto en las consultas	19
7.3 Impacto en los índices	19
8. Uso de herramientas de IA	20
9. Dificultades, decisiones y limitaciones conocidas	20
9.1 Dificultades y soluciones	20
9.2 Limitaciones y mejoras futuras	21
10. Conclusiones	21
11. Bibliografía, repositorio y video	21
Anexo A — Capturas de las mediciones de los índices base (TP1)	23

1. Resumen de la entrega y cobertura de los objetivos

Este informe acompaña la **primera entrega parcial** del Trabajo Práctico Integrador «Food Store», un sistema de gestión de pedidos de comida implementado íntegramente en **PostgreSQL 16+** con PL/pgSQL (sin interfaz ni API: toda la interacción es SQL). La entrega cubre las Unidades 1 a 3 de la materia: integridad, transacciones y concurrencia; optimización de consultas; e índices, vistas y objetos programables.

El proyecto se construyó en cinco trabajos prácticos sucesivos. Para esta entrega, el código quedó consolidado en dos archivos «fuente de verdad» (**schema.sql** y **objects.sql**), a los que se suman los datos (**data.sql**, **carga_masiva.sql**), las consultas (**queries.sql**) y los escenarios de transacciones (**transacciones.sql**). La tabla siguiente indica dónde se acredita cada uno de los nueve objetivos del checklist de la consigna.

#	Objetivo de la consigna	Dónde se acredita en este informe	Artefacto verificable
1	Modelo ER (entidades, atributos, claves, cardinalidad, participación)	§4.1, Figura 1 y tablas de entidades y relaciones	Diagrama ER
2	Paso de ER a relacional (1:N y N:M con tablas intermedias)	§4.2, modelo lógico y Figura 2	schema.sql · <i>detalle_pedido</i> resuelve la N:M
3	Normalización hasta 3FN/BCNF con justificación de dependencias funcionales	§4.3	Análisis por tabla; denormalización controlada de <i>subtotal</i> y <i>total</i>
4	DDL completo: tipos, PK/FK, restricciones e índices	§4.4 y §6.1	schema.sql (5 tablas, 3 ENUM, 6 índices)
5	DML y consultas: JOIN, agregación, subconsultas, GROUP BY/HAVING, ventana	§5.2, capturas 3 y 4	queries.sql
6	Vistas, funciones y procedimientos en PL/pgSQL	§6.2 a §6.4 y §4.6	objects.sql (5 vistas, 1 vista materializada, 1 función, 3 funciones de trigger, 1 procedimiento)
7	Reglas de negocio con CHECK, UNIQUE y triggers	§4.5 (incluye pruebas)	schema.sql y objects.sql
8	Transacciones: atomicidad, COMMIT/ROLLBACK, aislamiento, concurrencia	§4.6 (7 escenarios)	transacciones.sql y capturas
9	Borrado lógico y su impacto en consultas e índices	§7	Columna <i>eliminado</i> , vistas, índices parciales

Además, la consigna pide indicar qué se implementó en cada unidad, cómo se probó, qué resultados se obtuvieron, qué consultas se optimizaron (antes/después) y qué herramientas de IA se usaron. Esos puntos se responden en las secciones 4, 5 y 6 (una por unidad), y en la sección 8 (uso de IA).

Unidad	Qué se implementó	Cómo se probó	Resultado principal
1 · Integridad, transacciones y concurrencia	Modelo de 5 tablas; CHECK/UNIQUE; triggers de subtotal, total y transición de estado; <i>sp_crear_pedido</i> con bloqueo de filas	7 escenarios con dos sesiones concurrentes; pruebas de cada regla	Atomicidad, aislamiento y bloqueo demostrados; sin sobreventa
2 · Optimización de consultas	Carga masiva (~50 mil productos, 200 mil pedidos); índices candidatos; consultas analíticas	<i>EXPLAIN (ANALYZE, BUFFERS)</i> antes/después; pruebas de equivalencia con EXCEPT	De 3 índices evaluados en el TP3, 2 aceptados (~2,1x y ~1,7x) y 1 descartado; el covering de la competencia (1,52x) quedó documentado y no aplicado
3 · Índices, vistas y objetos programables	Índice de búsqueda por nombre; vista <i>v_usuario_seguro</i> ; vista materializada de facturación	Plan de indexado con 3 consultas; verificación de equivalencia de vistas; medición de la vista materializada	226x en búsqueda por nombre; ~3.700x en el reporte de facturación

2. Alcance, entorno y reproducción

2.1 Entorno y archivos del proyecto

Motor exigido: PostgreSQL 16 o superior (se usan tipos ENUM, TIMESTAMPTZ, columnas IDENTITY, tablas de transición en triggers, procedimientos con CALL y JSONB). Las pruebas y capturas del grupo se realizaron con **PostgreSQL 17 en pgAdmin 4**. Repositorio del proyecto: <https://github.com/marianoemir/TPI-BD-Food-Store>.

Archivo	Contenido	Orden
schema.sql	Tipos ENUM, 5 tablas, restricciones (PK, FK, UNIQUE, CHECK) y 6 índices. Consolida el esquema del TP1, los 2 CHECK del TP2 y los índices de los TP3/TP4/TP5.	1
objects.sql	4 vistas base, <i>v_usuario_seguro</i> , función <i>calcular_total_pedido</i> , triggers de subtotal/total/estado, <i>sp_crear_pedido</i> y la vista materializada con su índice único.	2
data.sql	Datos base de demostración (usuarios, categorías, productos y pedidos de ejemplo).	3
carga_masiva.sql	Carga sintética para mediciones: 20.006 usuarios, 50.016 productos, 200.006 pedidos y 400.013 detalles (contando los de data.sql).	4 (opcional)
queries.sql	Historias de usuario (DML) y consultas analíticas: JOIN, agregación, HAVING, subconsultas, ventana y equivalencias con EXCEPT.	5
transacciones.sql	Cuatro escenarios de transacciones y concurrencia; requiere dos sesiones abiertas a la vez (ejecución manual).	aparte

2.2 Verificación de reproducibilidad

Al armar este informe se ejecutó el proyecto completo sobre una base limpia de PostgreSQL 16.15 (verificación adicional, independiente de las corridas del grupo):

- *schema.sql* → *objects.sql* → *data.sql* corrieron sin errores y crearon 5 tablas, 6 índices (más el índice único de la vista materializada), 5 vistas y 1 vista materializada.
- *carga_masiva.sql* terminó en unos 15 segundos y dejó 20.006 usuarios, 50.016 productos, 200.006 pedidos y 400.013 detalles.
- *queries.sql* completo corrió sin errores (unos 164 segundos en ese entorno) y dejó exactamente los volúmenes de la Captura 2: **20.007 usuarios, 50.017 productos, 200.007 pedidos y 400.015 detalles** (las diferencias con el paso anterior son las altas de demostración que hace el propio script).

Detalle de orden importante: como *objects.sql* se ejecuta antes de cargar datos, la vista materializada *mv_facturacion_categoria_mes* se crea vacía (0 filas). Después de cargar *data.sql* o *carga_masiva.sql* hay que ejecutar **REFRESH MATERIALIZED VIEW CONCURRENTLY mv_facturacion_categoria_mes;** (ver §6.4). Tras la carga masiva, la vista quedó con 87 filas en la corrida de verificación.

2.3 Volumen de datos de prueba

Las mediciones de la Unidad 2 y 3 se hicieron sobre una copia de trabajo con carga masiva. La Captura 2 muestra el volumen real de filas sobre el que se corrieron las consultas finales.

Query		Query history	
1	SELECT		
2	(SELECT COUNT(*) FROM usuario) AS total_usuarios,		
3	(SELECT COUNT(*) FROM producto) AS total_productos,		
4	(SELECT COUNT(*) FROM pedido) AS total_pedidos,		
5	(SELECT COUNT(*) FROM detalle_pedido) AS total_detalles;		

Data Output		Messages	Notifications
SQL Showin			
total_usuarios	total_productos	total_pedidos	total_detalles
bigint	bigint	bigint	bigint
1	20007	50017	200007
			400015

Captura 2 — Volumen de datos: 20.007 usuarios, 50.017 productos, 200.007 pedidos, 400.015 detalles.

3. Marco teórico

El trabajo se sustenta en el **modelo relacional**: la información se organiza en tablas (relaciones) donde cada fila es una instancia de una entidad y cada columna un atributo; las tablas se vinculan mediante claves primarias (PK) y foráneas (FK) que garantizan la integridad referencial. La **normalización** estructura el esquema para eliminar redundancias y anomalías de inserción, actualización y borrado, verificando progresivamente la 1FN, 2FN, 3FN y la forma normal de Boyce-Codd (BCNF); a veces, y de forma justificada, se opta por una denormalización controlada cuando el beneficio de rendimiento supera el costo de mantener la redundancia. La integridad se garantiza con restricciones declarativas (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL) y, cuando la lógica lo requiere, con triggers y procedimientos que ejecutan las reglas de negocio dentro del propio motor.

El **control transaccional** se apoya en las propiedades ACID (atomicidad, consistencia, aislamiento y durabilidad). PostgreSQL implementa el aislamiento con **MVCC** (control de concurrencia multiversión): cada sentencia o transacción trabaja sobre una instantánea de los datos, por lo que las lecturas no bloquean a las escrituras. El nivel por defecto es READ COMMITTED (cada sentencia ve lo confirmado hasta ese instante); en REPEATABLE READ y SERIALIZABLE la transacción conserva la instantánea tomada en su primera sentencia. Para coordinar escrituras se usan bloqueos de fila (SELECT ... FOR UPDATE, UPDATE), y cuando dos transacciones se esperan mutuamente el motor detecta el **interbloqueo** (error 40P01) y aborta una.

En **optimización**, un índice B-Tree acelera una consulta cuando esta es lo bastante selectiva como para que leer el índice sea más barato que recorrer la tabla. Las herramientas de análisis son **EXPLAIN ANALYZE** (plan real con tiempos) y la opción **BUFFERS** (páginas leídas). Se usan variantes como índices **parciales** (con cláusula WHERE), de **expresión** (por ejemplo **lower(nombre)**) y **covering** (con INCLUDE, para lograr Index Only Scan). Las **vistas** encapsulan consultas y filtros comunes; las **vistas materializadas** guardan físicamente el resultado y se actualizan con REFRESH, cambiando frescura de datos por velocidad de lectura.

4. Unidad 1 — Modelo de datos, integridad, transacciones y concurrencia

4.1 Modelo Entidad-Relación (objetivo 1)

El sistema modela un negocio de comidas con cinco entidades: **categoría** (agrupa el menú), **producto** (catálogo con precio y stock), **usuario** (clientes y administradores), **pedido** (cabecera de la compra) y **detalle_pedido** (líneas de cada pedido).

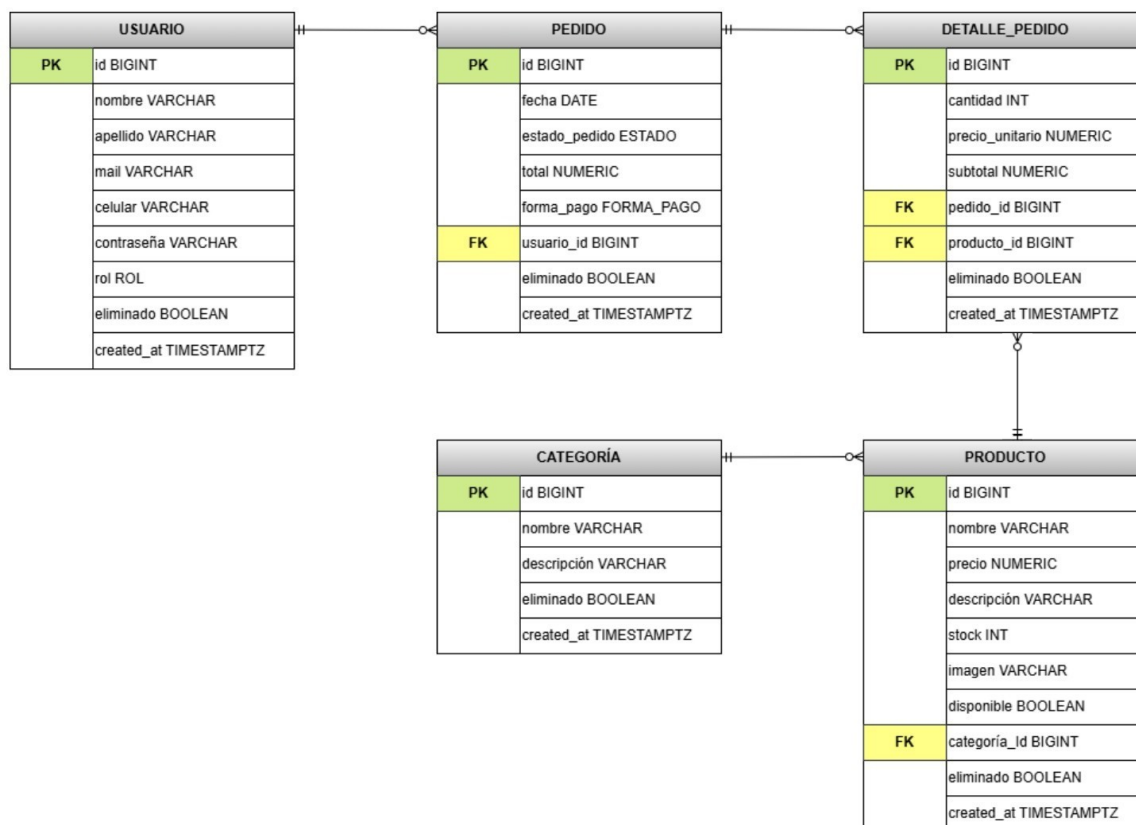


Figura 1 — Diagrama Entidad-Relación (notación pata de gallo). Se corrigió la notación de la relación PRODUCTO-DETALLE_PEDIDO respecto de la versión anterior: un producto figura en cero o muchos detalles, y cada detalle refiere exactamente a un producto.

Entidad	Clave primaria	Atributos	Claves foráneas / únicas
CATEGORÍA	id	nombre, descripcion, eliminado, created_at	UNIQUE(nombre)
PRODUCTO	id	nombre, precio, descripcion, stock, imagen, disponible, eliminado, created_at	categoría_id → categoría
USUARIO	id	nombre, apellido, mail, celular, contrasena, rol, eliminado, created_at	UNIQUE(mail)
PEDIDO	id	fecha, estado, total, forma_pago, eliminado, created_at	usuario_id → usuario
DETALLE_PEDIDO	id	cantidad, precio_unitario, subtotal, eliminado, created_at	pedido_id → pedido; producto_id → producto; UNIQUE(pedido_id, producto_id)

Relación	Cardinalidad	Participación
CATEGORÍA - PRODUCTO	1:N	Producto: total (categoría_id NOT NULL). Categoría: parcial (puede no tener productos).
USUARIO - PEDIDO	1:N	Pedido: total (usuario_id NOT NULL, «pedido siempre con usuario»). Usuario: parcial.
PEDIDO - DETALLE_PEDIDO	1:N	Detalle: total (pedido_id NOT NULL). Pedido: parcial en el modelo físico, porque <i>sp_crear_pedido</i> inserta primero la cabecera y luego los detalles dentro de la misma transacción.
PRODUCTO - DETALLE_PEDIDO	1:N	Detalle: total (producto_id NOT NULL). Producto: parcial (puede no haberse vendido nunca).

Relación	Cardinalidad	Participación
PEDIDO – PRODUCTO	N:M	Resuelta con la entidad asociativa DETALLE_PEDIDO, que aporta los atributos de la relación (cantidad, precio_unitario, subtotal).

Atributos repetidos en todas las entidades. *id* es una clave subrogada (**BIGINT GENERATED ALWAYS AS IDENTITY**) que simplifica los JOIN y evita claves compuestas; *eliminado* (**BOOLEAN DEFAULT FALSE**) implementa el borrado lógico (§7); y *created_at* (**TIMESTAMPZ DEFAULT now()**) registra el alta para auditoría y análisis temporal.

4.2 Paso a modelo relacional (objetivo 2)

Reglas de transformación aplicadas: cada entidad pasó a una tabla con su clave primaria; cada relación 1:N se resolvió llevando la clave foránea al lado «muchos»; y la relación N:M entre pedido y producto se resolvió con la tabla intermedia *detalle_pedido*, que contiene las dos claves foráneas y una restricción **UNIQUE(pedido_id, producto_id)** para no repetir un producto dentro de un pedido.

```

categoria(id, nombre, descripcion, eliminado, created_at)
producto(id, nombre, precio, descripcion, stock, imagen, disponible, categoria_id -> categoria, eliminado, created_at)
usuario(id, nombre, apellido, mail, celular, contrasena, rol, eliminado, created_at)
pedido(id, fecha, estado, total, forma_pago, usuario_id -> usuario, eliminado, created_at)
detalle_pedido(id, cantidad, precio_unitario, subtotal, pedido_id -> pedido, producto_id -> producto, eliminado, created_at)

```

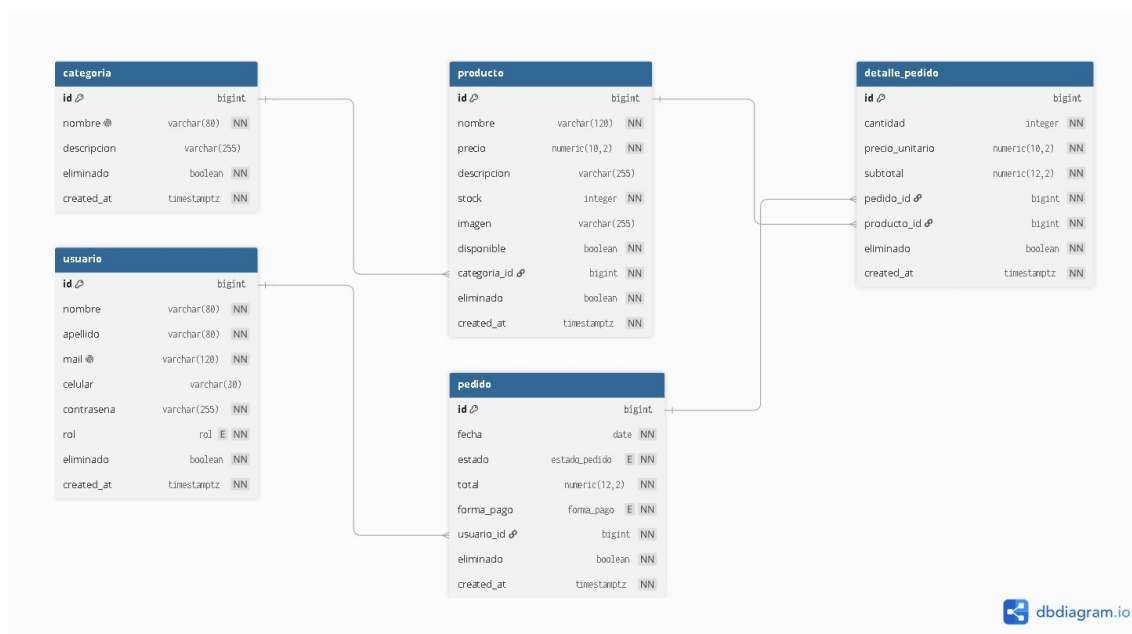


Figura 2 — Modelo físico generado a partir de schema.sql (tipos, NOT NULL, PK y FK).

4.3 Normalización hasta 3FN/BCNF (objetivo 3)

1FN. Todos los atributos son atómicos y no hay grupos repetitivos. La alternativa de guardar los productos de un pedido como una lista dentro de *pedido* (por ejemplo, ids separados por comas) violaría la 1FN y haría imposible declarar restricciones básicas (UNIQUE por producto, CHECK de cantidad) o indexar y unir eficientemente; por eso existe *detalle_pedido*.

2FN. Las cinco tablas tienen clave primaria simple (*id*), por lo que no puede haber dependencias parciales: se cumple de forma trivial. Aun considerando la clave natural (*pedido_id*, *producto_id*) de *detalle_pedido*, sus atributos dependen de la clave completa.

3FN/BCNF. Se verificó que todo determinante de una dependencia funcional no trivial sea una superclave:

Tabla	Claves candidatas	Dependencias funcionales	Resultado
categoria	id, nombre	$id \rightarrow \{\text{nombre, descripcion, eliminado, created_at}\}$; $\text{nombre} \rightarrow id$	BCNF
producto	id	$id \rightarrow \{\text{nombre, precio, descripcion, stock, imagen, disponible, categoria_id, ...}\}$	BCNF

Tabla	Claves candidatas	Dependencias funcionales	Resultado
usuario	id, mail	id → todos; mail → todos. Sin otras dependencias (nombre no determina apellido, celular no determina rol)	BCNF (dos claves candidatas verificadas)
pedido	id	id → {fecha, estado, total, forma_pago, usuario_id, ...}	BCNF, con redundancia inter-tabla en <i>total</i>
detalle_pedido	id, (pedido_id, producto_id)	id → todos; (pedido_id, producto_id) → todos; id → {cantidad, precio_unitario} → subtotal	Excepción documentada: <i>subtotal</i> (transitiva)

Denormalización controlada: subtotal y total

Hay dos atributos calculables. En *detalle_pedido*, $subtotal = cantidad \times precio_unitario$ es una dependencia transitiva intra-relación (viola 3FN en sentido estricto). En *pedido*, $total = \sum subtotales$ es una redundancia inter-tabla, porque se puede recalcular desde *detalle_pedido*. Se decidió **materializar ambos** por dos motivos:

- **Rendimiento de lectura:** los listados de pedidos y los reportes (facturación por categoría, ranking de gasto) se consultan mucho más que lo que se modifican los detalles; recalcular la suma en cada lectura sería costoso con cientos de miles de pedidos.
- **Historial congelado:** *precio_unitario* se fija en el momento de la venta y no acompaña a *producto.precio*; por lo tanto *subtotal* es un hecho histórico, no un valor derivable del estado actual.

La consistencia no queda en manos de la aplicación: la garantiza el motor con los triggers `trg_subtotal`, `trg_total_ins` y `trg_total_upd` (§6.3), de modo que ningún INSERT o UPDATE puede dejar ambos valores desincronizados.

4.4 DDL completo (objetivo 4)

El DDL está en **schema.sql**. Resumen de decisiones de tipos y restricciones:

Aspecto	Definición
Tipos enumerados	<code>rol</code> (ADMIN, USUARIO), <code>estado_pedido</code> (PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO), <code>forma_pago</code> (TARJETA, TRANSFERENCIA, EFECTIVO)
Claves primarias	<code>id</code> BIGINT GENERATED ALWAYS AS IDENTITY en las 5 tablas
Claves foráneas	<code>producto.categoria_id</code> , <code>pedido.usuario_id</code> , <code>detalle_pedido.pedido_id</code> (ON DELETE RESTRICT), <code>detalle_pedido.producto_id</code> ; todas NOT NULL
UNIQUE	<code>categoria.nombre</code> ; <code>usuario.mail</code> ; <code>detalle_pedido(pedido_id, producto_id)</code>
CHECK	<code>precio ≥ 0</code> ; <code>stock ≥ 0</code> ; <code>cantidad > 0</code> ; <code>precio_unitario ≥ 0</code> ; <code>subtotal ≥ 0</code> ; <code>total ≥ 0</code> ; formato de mail; <code>fecha ≤ CURRENT_DATE</code>
Precisión numérica	NUMERIC(10,2) para precios; NUMERIC(12,2) para subtotales y totales
Índices	6 índices B-Tree (4 parciales, uno de ellos de expresión); ver §6.1

```
CREATE TABLE usuario (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL, apellido VARCHAR(80) NOT NULL,
  mail VARCHAR(120) NOT NULL UNIQUE, celular VARCHAR(30), contrasena VARCHAR(255) NOT NULL,
  rol rol NOT NULL DEFAULT 'USUARIO',
  eliminado BOOLEAN NOT NULL DEFAULT FALSE, created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT chk_usuario_mail_formato CHECK (mail ~ '^([A-Za-z0-9._%+-])+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
);
CREATE TABLE pedido (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  fecha DATE NOT NULL DEFAULT CURRENT_DATE CHECK (fecha <= CURRENT_DATE),
  estado estado_pedido NOT NULL DEFAULT 'PENDIENTE',
  total NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (total >= 0),
  forma_pago forma_pago NOT NULL,
  usuario_id BIGINT NOT NULL REFERENCES usuario(id),
  eliminado BOOLEAN NOT NULL DEFAULT FALSE, created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Corrección respecto del DDL de referencia. El fragmento provisto por la cátedra definía la clave primaria de *categoria* con el tipo *BIGINT* (doble «G»), que no existe en PostgreSQL. Se detectó al ejecutar el script y se corrigió a *BIGINT*, coherente con el resto de las tablas.

4.5 Reglas de negocio (objetivo 7)

Las reglas se implementan con restricciones declarativas cuando es posible, y con triggers o el procedimiento almacenado cuando la regla no se puede expresar como restricción simple. En negrita, las agregadas después del diseño inicial (TP2).

Regla de negocio	Mecanismo en la base de datos	Origen
Precio y stock no negativos	CHECK (precio >= 0), CHECK (stock >= 0)	TP1
Cantidad de detalle > 0	CHECK (cantidad > 0)	TP1
Mail de usuario único	UNIQUE (mail)	TP1
Mail con formato válido	CHECK chk_usuario_mail_formato (expresión regular)	TP2
Fecha de pedido no futura	CHECK (fecha <= CURRENT_DATE) en pedido	TP2
Un pedido CONFIRMADO no vuelve a PENDIENTE	Trigger trg_validar_estado_pedido (BEFORE UPDATE OF estado)	TP2
No repetir producto en un mismo pedido	UNIQUE (pedido_id, producto_id)	TP1
Pedido siempre con usuario	usuario_id NOT NULL + FOREIGN KEY	TP1
subtotal = cantidad × precio_unitario (precio congelado)	Trigger BEFORE INSERT/UPDATE en detalle_pedido	TP1
total del pedido = Σ subtotales vigentes	Triggers AFTER ... FOR EACH STATEMENT en detalle_pedido	TP1
No vender sin stock ni producto no disponible	Validación en sp_crear_pedido con SELECT ... FOR UPDATE y descuento de stock	TP1
Baja lógica (nunca DELETE físico)	Columna eliminado; FK con ON DELETE RESTRICT	TP1

Pruebas de las reglas agregadas en el TP2

Para cada regla se ejecutó una sentencia que la viola y otra que la respeta (verificación sobre PostgreSQL 16.15 con *data.sql* cargado). El motor rechazó las violaciones con el mensaje esperado:

Prueba	Sentencia	Resultado observado
Mail con formato inválido	<code>INSERT INTO usuario(...) VALUES ('Test', 'Mail', 'esto-no-es-un-mail', 'x')</code>	ERROR: viola la restricción CHECK «chk_usuario_mail_formato»
Mail válido	<code>... mail = 'test.ok@example.com'</code>	Inserción correcta (id 8)
Mail duplicado	Mismo mail insertado por segunda vez	ERROR: viola la restricción UNIQUE «usuario_mail_key»
Fecha futura	<code>INSERT INTO pedido(fecha, ...) VALUES (CURRENT_DATE + 1, ...)</code>	ERROR: viola la restricción CHECK «pedido_fecha_check»
CONFIRMADO → PENDIENTE	<code>UPDATE pedido SET estado='PENDIENTE' WHERE id = 8 -- estaba CONFIRMADO</code>	ERROR: No se permite cambiar el estado de un pedido de CONFIRMADO a PENDIENTE (Pedido ID: 8)
CONFIRMADO → TERMINADO	<code>UPDATE pedido SET estado='TERMINADO' WHERE id = 8</code>	UPDATE 1 (transición permitida)
Precio negativo / cantidad 0	<code>INSERT ... precio = -1 y INSERT ... cantidad = 0</code>	ERROR: CHECK «producto_precio_check» y «detalle_pedido_cantidad_check»
Producto repetido en un pedido	Segundo detalle del mismo (pedido_id, producto_id)	ERROR: viola UNIQUE «detalle_pedido_pedido_id_producto_id_key»

4.6 Transacciones y concurrencia (objetivo 8)

El procedimiento transaccional sp_crear_pedido

sp_crear_pedido(p_usuario_id, p_forma_pago, p_items JSONB) da de alta un pedido y todos sus detalles como una unidad. Valida que el usuario exista y no esté eliminado; inserta la cabecera; y por cada ítem del JSON bloquea la fila del producto con `SELECT ... FOR UPDATE`, valida que exista, esté disponible y tenga stock, inserta el detalle y descuenta el stock.

```
FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
    v_producto_id := (v_item->>'producto_id')::BIGINT; v_cantidad := (v_item->>'cantidad')::INTEGER;
    SELECT stock, disponible INTO v_stock, v_disponible
    FROM producto WHERE id = v_producto_id AND eliminado = FALSE FOR UPDATE; -- bloquea la fila
    IF NOT FOUND THEN RAISE EXCEPTION 'Producto % inexistente o eliminado', v_producto_id; END IF;
    IF NOT v_disponible THEN RAISE EXCEPTION 'Producto % no disponible', v_producto_id; END IF;
```

```

IF v_stock < v_cantidad THEN
    RAISE EXCEPTION 'Stock insuficiente (producto %): hay %, se piden %', v_producto_id, v_stock, v_cantidad;
END IF;
INSERT INTO detalle_pedido(pedido_id, producto_id, cantidad) VALUES (v_pedido_id, v_producto_id, v_cantidad);
UPDATE producto SET stock = stock - v_cantidad WHERE id = v_producto_id;
END LOOP;

```

Dos decisiones de diseño: (1) el bloqueo con *FOR UPDATE* serializa el acceso concurrente al mismo producto y evita la sobreventa; (2) un PROCEDURE de PostgreSQL **no abre su propia transacción**, se ejecuta dentro de la del llamador, por lo que una excepción no capturada aborta toda la transacción envolvente y no queda ni el pedido ni ningún detalle. El rollback no lo gestiona el procedimiento, sino que es consecuencia de propagar la excepción.

Escenarios ejecutados

Los escenarios 1 a 4 están en **transacciones.sql** (dos pestañas de Query Tool contra la misma base para simular dos operadores). Los escenarios 5 a 7 provienen del trabajo de concurrencia del TP2 (*informe_concurrencia*), donde además se contrastó la explicación de la IA con lo observado en el motor real.

#	Escenario	Qué se hizo	Qué se observó	Aislamiento / mecanismo
1	Atomicidad	CALL sp_crear_pedido con una cantidad que excede el stock del producto 7 (9999 unidades)	ERROR «Stock insuficiente (producto 7)»; no quedó registrado ningún pedido ni detalle	Propagación de excepción; transacción implícita
2	COMMIT vs ROLLBACK	UPDATE de stock del producto 7 en un BEGIN...COMMIT y otro en BEGIN...ROLLBACK	El cambio del COMMIT persiste; el del ROLLBACK se deshace y el stock vuelve a 90	Transacción manual
3	Lectura no repetible	Sesión A lee dos veces el stock del producto 7; entre lecturas, B descuenta 5 y confirma	READ COMMITTED: 90 → 85 (cambia). SERIALIZABLE: 85 → 85 (no cambia)	READ COMMITTED vs SERIALIZABLE
4	Bloqueo FOR UPDATE	Dos sesiones piden 8 unidades del producto 3 (stock 15) con sp_crear_pedido	B esperó más de 30 s; tras el COMMIT de A vio stock 7 y fue rechazada: sin sobreventa	SELECT ... FOR UPDATE
5	Lectura fantasma (TP2)	A cuenta productos de la categoría 1; B inserta uno y confirma; A repite el COUNT	READ COMMITTED: 5 → 6. REPEATABLE READ: 6 → 6	READ COMMITTED vs REPEATABLE READ
6	Interbloqueo (TP2)	A bloquea fila 1 y pide la 2; B bloquea la 2 y pide la 1 (SELECT ... FOR UPDATE cruzado)	PostgreSQL abortó automáticamente a la Sesión B con «40P01: deadlock detected». Con orden ascendente de ids no hay ciclo	Detector de deadlocks; orden consistente de bloqueo
7	Espera por bloqueo de fila (TP2)	A hace UPDATE del precio del producto 1 sin confirmar; B intenta otro UPDATE de la misma fila	B quedó en espera hasta el COMMIT de A y recién entonces completó, sin pérdida de actualizaciones	Bloqueo de fila (evita lost update)

Escenario 1 — Atomicidad

Data Output	Messages	Notifications
<pre> ERROR: Stock insuficiente (producto 7): hay 80, se piden 9999 CONTEXT: función PL/pgSQL sp_crear_pedido(bigint,forma_pago,jsonb) en la línea 38 en RAISE SQL state: P0001 </pre>		
Total rows: 1	Query complete 00:00:00.062	CRLF Ln 1, Col 1

Escenario 1 — El procedimiento rechaza la venta imposible antes de dejar rastro en la base.

Escenario 2 — COMMIT vs ROLLBACK

Paso	Stock	¿Persiste?
Valor inicial	94	—
Dentro de la transacción, antes de COMMIT	90	No todavía
Después de COMMIT	90	Sí

Paso	Stock	¿Persiste?
Dentro de la transacción, antes de ROLLBACK	89	No todavía
Después de ROLLBACK	90	No, se revirtió

Escenario 3 — Aislamiento: lectura no repetible

Con READ COMMITTED (nivel por defecto) la segunda lectura de A, sin que A hubiera confirmado nada, devolvió 85: dos SELECT idénticos en la misma transacción dieron valores distintos. Repetida la prueba con **BEGIN; SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;** en A (la instrucción debe ir dentro de la transacción, inmediatamente después de BEGIN), la primera lectura dio 85, B volvió a confirmar un descuento (stock → 80) y la segunda lectura de A siguió dando 85: la transacción trabaja con la instantánea tomada en su primera sentencia y no ve cambios confirmados por otros hasta terminar. Su COMMIT no tuvo conflicto porque A solo había leído.

Escenario 4 — Bloqueo FOR UPDATE frente a la sobreventa

La Sesión A ejecutó el CALL con éxito y quedó con el bloqueo de la fila del producto 3 sin hacer COMMIT. La Sesión B, al comprar el mismo producto, quedó esperando («Waiting for the query to complete...» durante más de 30 segundos) sin leer nada. Al confirmar A, B vio el stock ya actualizado (7) y el procedimiento respondió *Stock insuficiente (producto 3): hay 7, se piden 8*. Stock final: 7. Sin el bloqueo, ambas habrían leído 15 unidades y descontado 8 cada una, dejando un stock negativo o inconsistente.

Data Output Messages Notifications CALL Query returned successfully in 142 msec. Total rows: Query complete 00:00:00.142 CRLF Ln 1, Col 1	Data Output Messages Notifications Waiting for the query to complete... 00:00:33.537 Total rows: CRLF Ln 1, Col 1
---	--

Escenario 4 (1/2) — Sesión A: CALL exitoso (izquierda). Sesión B: esperando el bloqueo (derecha).

Data Output Messages Notifications COMMIT Query returned successfully in 37 msec. Total rows: Query complete 00:00:00.097 CRLF Ln 3, Col 1	Data Output Messages Notifications ERROR: Stock insuficiente (producto 3): hay 7, se piden 8 CONTEXT: función PL/pgSQL sp_crear_pedido(origen,forma_pago,jsonb) en la línea 38 en RAISE SQL state: P0001 Total rows: Query complete 00:00:06.930 CRLF Ln 1, Col 1
--	--

Escenario 4 (2/2) — Sesión A: COMMIT (izquierda). Sesión B: rechazo de negocio «hay 7, se piden 8» (derecha).

Escenarios 5 a 7 — Fantasma, interbloqueo y espera de fila

Fantasma. Con READ COMMITTED cada SELECT toma una instantánea nueva, por eso el COUNT pasó de 5 a 6 cuando B confirmó un INSERT. Con REPEATABLE READ (donde la instantánea se fija al inicio) el conteo se mantuvo en 6 aunque B insertó otra fila. **Interbloqueo.** Al cruzar el orden de bloqueo entre A y B se forma una espera circular; tras el *deadlock_timeout* (1 s por defecto) el detector elige una víctima y le devuelve 40P01. Subir el nivel de aislamiento no lo evita; lo evita adquirir los bloqueos siempre en el mismo orden (ids ascendentes), mantener transacciones cortas y reintentar ante 40P01. **Espera de fila.** Un UPDATE toma un bloqueo exclusivo de fila; otra transacción que quiera modificarla espera al COMMIT o ROLLBACK de la primera, lo que previene la pérdida de actualizaciones.

Fenómeno	READ COMMITTED	REPEATABLE READ / SERIALIZABLE	Evidencia
Lectura no repetible	Ocurre (90 → 85)	No ocurre en SERIALIZABLE (85 → 85)	Escenario 3
Lectura fantasma	Ocurre (COUNT 5 → 6)	No ocurre en REPEATABLE READ (COUNT 6 → 6)	Escenario 5
Actualización perdida / sobreventa	Se evita con bloqueo de fila (UPDATE, FOR UPDATE)	Ídem	Escenarios 4 y 7
Interbloqueo	Detectado y abortado (40P01)	Detectado y abortado (40P01)	Escenario 6

5. Unidad 2 — Consultas y optimización

5.1 Datos de prueba

Para medir con volúmenes realistas se cargaron datos sintéticos con **carga_masiva.sql** sobre una copia de trabajo: **20.007 usuarios, 50.017 productos, 200.007 pedidos y 400.015 detalles** (Captura 2, §2.3). En la primera versión del script, las fechas de los pedidos no caían en el rango que usa una de las consultas medidas y el paso de detalles asumía ids de pedido incorrectos; se corrigió, se recreó la copia de trabajo y se **volvió a medir** todo. Por eso los números de este informe corresponden a la segunda medición y pueden diferir levemente de una corrida anterior (la carga usa `random()`); las conclusiones sobre qué índices ayudan y cuáles no se mantuvieron.

5.2 Consultas: JOIN, agregación, subconsultas y funciones de ventana (objetivo 5)

queries.sql tiene dos secciones: la 1 implementa las historias de usuario (altas, ediciones, bajas lógicas, listados por vista y creación de pedidos con CALL); la 2 contiene las consultas analíticas. La cobertura de técnicas es la siguiente:

Técnica	Consulta (queries.sql)
JOIN (INNER / LEFT) y agregación con GROUP BY	Top 5 productos más vendidos (SUM sobre detalle_pedido ⋈ producto); facturación mensual por categoría (4 tablas: detalle, pedido, producto, categoría); productos sin ventas (LEFT JOIN + IS NULL)
HAVING	Subconsulta agrupada con <code>HAVING COUNT(p.id) >= 1</code> en el ranking de usuarios (Opción C)
Subconsultas	Correlacionada en WHERE (productos con precio mayor al promedio de su categoría); subconsulta agrupada en FROM; NOT EXISTS en las altas idempotentes de las historias de usuario
Funciones de ventana	<code>DENSE_RANK() OVER (ORDER BY SUM(p.total) DESC, u.id ASC)</code> : ranking de usuarios por gasto, resuelto de tres formas (JOIN directo, CTE y subconsulta con HAVING)
Pruebas de equivalencia	Cada forma alternativa se compara contra otra con EXCEPT (2.2 y 2.3)

```
-- Ranking de usuarios por gasto (Opción A: JOIN directo + función de ventana)
SELECT u.nombre || ' ' || u.apellido AS nombre_completo,
       SUM(p.total) AS total_gastado,
       DENSE_RANK() OVER (ORDER BY SUM(p.total) DESC, u.id ASC) AS puesto
FROM usuario u
INNER JOIN pedido p ON p.usuario_id = u.id AND p.eliminado = FALSE
WHERE u.eliminado = FALSE
GROUP BY u.id, u.nombre, u.apellido;
```

```

1 SELECT
2     u.nombre || ' ' || u.apellido AS nombre_completo,
3     SUM(p.total) AS total_gastado,
4     DENSE_RANK() OVER (ORDER BY SUM(p.total) DESC, u.id ASC) AS puesto
5 FROM usuario u
6 INNER JOIN pedido p ON p.usuario_id = u.id AND p.eliminado = FALSE
7 WHERE u.eliminado = FALSE
8 GROUP BY u.id, u.nombre, u.apellido
9 LIMIT 15;

```

Data Output Messages Notifications

Showing rows: 1 to 15

	nombre_completo text	total_gastado numeric	puesto bigint
1	Usuario_3184 Apellido_3184	213341.94	1
2	Usuario_13184 Apellido_131...	213341.94	2
3	Usuario_1424 Apellido_1424	212047.26	3
4	Usuario_11424 Apellido_114...	212047.26	4
5	Usuario_8356 Apellido_8356	209053.88	5
6	Usuario_18356 Apellido_183...	209053.88	6
7	Usuario_2431 Apellido_2431	208176.54	7
8	Usuario_12431 Apellido_124...	208176.54	8
9	Usuario_1944 Apellido_1944	205553.52	9
10	Usuario_11944 Apellido_119...	205553.52	10
11	Usuario_7755 Apellido_7755	202694.12	11
12	Usuario_17755 Apellido_177...	202694.12	12
13	Usuario_7786 Apellido_7786	202682.54	13
14	Usuario_17786 Apellido_177...	202682.54	14
15	Usuario_1532 Apellido_1532	202203.40	15

Captura 3 — Ranking de usuarios con DENSE_RANK() sobre 200.007 pedidos (15 primeras filas, 0,148 s).

Equivalencia de resoluciones. La Opción A (JOIN directo) y la B (CTE) del ranking se compararon con **A EXCEPT B**: el resultado fue **0 filas** (Captura 4). Como EXCEPT solo mira en un sentido, en la verificación adicional de este informe se corrió también **B EXCEPT A** y dio 0 filas sobre los 19.998 usuarios rankeados; la misma doble comprobación dio 0 en la consulta de productos con precio mayor al promedio de su categoría (subconsulta correlacionada vs. JOIN con subconsulta agrupada).

```

1 WITH totales_usuarios AS (
2     SELECT u.id AS usuario_id, u.nombre || ' ' || u.apellido AS nombre_completo, SUM(p.total) AS total_gastado
3     FROM usuario u INNER JOIN pedido p ON p.usuario_id = u.id AND p.eliminado = FALSE WHERE u.eliminado = FALSE GROU
4 )
5 SELECT u.nombre || ' ' || u.apellido AS nombre_completo, SUM(p.total) AS total_gastado, DENSE_RANK() OVER (ORDER BY
6 FROM usuario u INNER JOIN pedido p ON p.usuario_id = u.id AND p.eliminado = FALSE WHERE u.eliminado = FALSE GROUP BY
7 EXCEPT
8 SELECT nombre_completo, total_gastado, DENSE_RANK() OVER (ORDER BY total_gastado DESC, usuario_id ASC) AS puesto FROM

```

Data Output Messages Notifications

Showing rows: 1 to 15

	nombre_completo text	total_gastado numeric	puesto bigint
1	Usuario_3184 Apellido_3184	213341.94	1
2	Usuario_13184 Apellido_131...	213341.94	2
3	Usuario_1424 Apellido_1424	212047.26	3
4	Usuario_11424 Apellido_114...	212047.26	4
5	Usuario_8356 Apellido_8356	209053.88	5
6	Usuario_18356 Apellido_183...	209053.88	6
7	Usuario_2431 Apellido_2431	208176.54	7
8	Usuario_12431 Apellido_124...	208176.54	8
9	Usuario_1944 Apellido_1944	205553.52	9
10	Usuario_11944 Apellido_119...	205553.52	10
11	Usuario_7755 Apellido_7755	202694.12	11
12	Usuario_17755 Apellido_177...	202694.12	12
13	Usuario_7786 Apellido_7786	202682.54	13
14	Usuario_17786 Apellido_177...	202682.54	14
15	Usuario_1532 Apellido_1532	202203.40	15

Captura 4 — Comprobación de equivalencia con EXCEPT: 0 filas de diferencia entre las dos formas del ranking.

Observación sobre el desempate. Al incluir *u.id* en el ORDER BY de la ventana, dos usuarios con igual gasto reciben puestos consecutivos distintos (en la Captura 3, 213.341,94 ocupa los puestos 1 y 2), con lo que DENSE_RANK se comporta como ROW_NUMBER. Es intencional para obtener un orden determinista; si se quisiera que los empates compartan puesto, alcanza con quitar *u.id* del ORDER BY de la ventana.

5.3 Metodología de optimización

- Cada candidato se documentó con una lista de control: consulta, frecuencia de uso, plan «antes», selectividad estimada, índice propuesto, plan «después», mejora y decisión (aceptado o descartado).

- Los planes se obtuvieron con **EXPLAIN (ANALYZE, BUFFERS)** sobre la copia con carga masiva; después de crear cada índice se ejecutó **ANALYZE** para actualizar estadísticas.
- Criterio de aceptación: el optimizador debe **adoptar** el índice y la mejora debe ser sustancial en tiempo y en páginas leídas. Se descartó todo índice que no fue adoptado o cuya mejora no compensaba el costo de mantenerlo, sobre todo en tablas de escritura frecuente como *pedido* y *detalle_pedido*.
- La IA propuso estrategias en varias etapas, pero ninguna se aplicó sin leerla y validarla contra el plan real (ver §8).

5.4 Resultados antes/después

A. Consultas medidas en el TP3 (índices aplicados en el TP4)

Consulta	Plan antes	Cambio aplicado	Plan después	Mejora
Historial de pedidos por rango de fechas y estado	Seq Scan on pedido (Hash Join) costo 825,13..8303,55 · buffers 3.932 27,910 ms	idx_pedido_fecha_estado_vigente pedido(fecha, estado) WHERE eliminado = FALSE	Bitmap Heap Scan (Hash Join) costo 1969,51..6256,51 · buffers 2.125 13,318 ms	~2,1x más rápido; buffers a casi la mitad
Búsqueda de productos por categoría y precio	Sort (quicksort, 1.007 kB) + Bitmap Heap Scan con el índice existente buffers 920 · 7,750 ms	idx_producto_categoria_precio_vigente producto(categoria_id, precio DESC) WHERE eliminado = FALSE	Sin cambio: el optimizador ignoró el índice nuevo 7,750 ms	Descartado: el Sort en memoria de ~10.000 filas resultó más barato
Ranking de clientes por total gastado	Seq Scan on pedido + Hash Join + HashAggregate costo 8698,53 · buffers 4.307 88,860 ms	idx_pedido_usuario_total_estado pedido(usuario_id, total) WHERE eliminado = FALSE AND estado IN (CONFIRMADO, TERMINADO)	Index Only Scan (Heap Fetches: 0) + Hash Join + HashAggregate costo 5643,71 · buffers 845 52,341 ms	~1,7x más rápido; buffers -80 %

B. Plan de indexado de la Unidad 3 (TP5): tres consultas de queries.sql

Consulta	Selectividad	Índice probado	Antes → después	Decisión
Top 5 productos más vendidos	~100 % (agrupa todo detalle_pedido)	idx_detalle_pedido_producto_id	726,968 ms → 585,766 ms. Plan idéntico: el optimizador no adoptó el índice	Descartado (la diferencia es variación entre corridas)
Pedidos con total superior al promedio	50,05 % (medida exacta)	idx_pedido_total	373,099 ms → 300,029 ms (~19 %). Adoptado, pero lee ~la mitad del heap	Descartado: mejora marginal, costo sobre tabla de escritura frecuente
Búsqueda de productos por nombre (LIKE de prefijo, sin distinguir mayúsculas)	~0,002 % (1 de 50.016)	idx_producto_nombre_lower_vigente	50,191 ms → 0,222 ms; buffers 910 → 3; Seq Scan → Bitmap Heap Scan	Aceptado: ~226x

```
-- Consulta 3, índice aceptado (necesita text_pattern_ops para soportar LIKE de prefijo)
CREATE INDEX idx_producto_nombre_lower_vigente
ON producto (lower(nombre) text_pattern_ops) WHERE eliminado = FALSE;
```

```
-- Plan después (rango reescrito por el optimizador):
Bitmap Index Scan on idx_producto_nombre_lower_vigente
  Index Cond: ((lower(nombre) ~>= 'pizza') AND (lower(nombre) ~<= 'pizzb'))
Execution Time: 0.222 ms      -- antes: Seq Scan, 50.191 ms
```

Evidencia en la práctica. La Captura 5 muestra el mismo índice en uso con otro término de búsqueda (0,047 ms). La consulta no está escrita en *queries.sql*, así que se transcribe para poder replicarla; el «antes» de referencia es el de la búsqueda «pizza%» de la tabla anterior.

```
EXPLAIN ANALYZE
SELECT id, nombre, precio
FROM producto
WHERE lower(nombre) LIKE 'hamb%' AND eliminado = FALSE;
```

1	EXPLAIN ANALYZE
2	SELECT id, nombre, precio
3	FROM producto
4	WHERE lower(nombre) LIKE 'hamb%' AND eliminado = FALSE;

Data Output	Messages	Notifications
Showing rows: 1 to 8		
QUERY PLAN		
1	Bitmap Heap Scan on producto (cost=10.98..570.21 rows=250 width=28) (actual time=0.029..0.031 rows=2 loops=1)	
2	Recheck Cond: (NOT eliminado)	
3	Filter: (lower((nombre)::text) ~~ 'hamb%':text)	
4	Heap Blocks: exact=1	
5	-> Bitmap Index Scan on idx_producto_nombre_lower_vigente (cost=0.00..10.91 rows=250 width=0) (actual time=0.008..0.008 rows=2 loops=1)	
6	Index Cond: ((lower((nombre)::text) ~~ 'hamb%':text) AND (lower((nombre)::text) ~~ 'hamb%':text))	
7	Planning Time: 0.356 ms	
8	Execution Time: 0.047 ms	

Captura 5 — EXPLAIN ANALYZE: Bitmap Index Scan sobre idx_producto_nombre_lower_vigente, 0,047 ms.

C. Vista materializada de facturación

El reporte de facturación total por categoría y mes une cuatro tablas base (categoría, producto, detalle_pedido, pedido), filtra por estado y borrado lógico y agrega cientos de miles de filas. Ejecutado sobre las tablas base tardó **164,285 ms**; consultado sobre la vista materializada, **0,044 ms** (reducción de ~99,97 %, unas **3.700 veces** más rápido). Detalle en §6.4.

D. Ejercicio de competencia: índice covering

En el TP3 se propuso a la IA optimizar una consulta entregada por la cátedra sobre *producto* (categoría 1, disponible, precio entre 1.000 y 3.000, ordenada por precio descendente). El plan «antes» usaba *idx_producto_categoria_id* (Bitmap Heap Scan que trae 10.006 filas y descarta 5.560 por filtro) más un Sort en memoria: **14,185 ms**. El índice covering ganador redujo el tiempo a **9,311 ms (1,52x)** y eliminó el Filter y el Sort (Index Only Scan, Heap Fetches: 2).

```
CREATE INDEX idx_producto_cat_disp_precio_cov
ON producto (categoria_id, disponible, precio DESC) INCLUDE (id, nombre, stock);
```

Estado en la base final: evaluado, no aplicado. Este índice ganó la comparación, pero **no forma parte del esquema final**: en *schema.sql* permanece como comentario y no se crea. Por eso el informe lo cuenta como evidencia del proceso de evaluación y no como índice en uso. Una razón técnica a favor de esa decisión es que incluye *stock*, que *sp_crear_pedido* actualiza en cada venta, por lo que el índice se tendría que mantener en cada operación de escritura más frecuente del sistema. Si el grupo decidiera activarlo, bastaría descomentar la sentencia en *schema.sql* y actualizar esta tabla y la del §6.1.

5.5 Resumen de decisiones sobre índices

Índice	Definición	Resultado medido	Decisión	En schema.sql
idx_producto_categoria_id	producto(categoria_id)	Con 5.000 filas: costo 124,50 → 71,34; el tiempo no mejoró (1,857 → 4,604 ms, con lecturas de disco en la corrida con índice). Con 50.000 filas el planificador lo usa (plan «antes» de la competencia)	Aplicado	Sí
idx_pedido_usuario_id	pedido(usuario_id)	Seq Scan → Bitmap Heap Scan; costo 44,00 → 14,85; 0,665 → 0,160 ms	Aplicado	Sí

Índice	Definición	Resultado medido	Decisión	En schema.sql
<code>idx_producto_nombre_vigente</code>	producto(nombre) WHERE eliminado = FALSE	Seq Scan → Index Scan; 1,552 → 0,111 ms (búsqueda exacta)	Aplicado	Sí
<code>idx_pedido_fecha_estado_vigente</code>	pedido(fecha, estado) parcial	27,910 → 13,318 ms (~2,1x)	Aplicado	Sí
<code>idx_pedido_usuario_total_estado</code>	pedido(usuario_id, total) parcial	88,860 → 52,341 ms (~1,7x); Index Only Scan	Aplicado	Sí
<code>idx_producto_nombre_lower_vigente</code>	producto(lower(nombre) text_pattern_ops) parcial	50,191 → 0,222 ms (~226x)	Aplicado	Sí
<code>idx_producto_categoria_precio_vigente</code>	producto(categoria_id, precio DESC) parcial	Sin cambio de plan	Descartado	No
<code>idx_detalle_pedido_producto_id</code>	detalle_pedido(producto_id)	Plan idéntico, no adoptado	Descartado	No
<code>idx_pedido_total</code>	pedido(total) parcial	~19 % (373 → 300 ms)	Descartado	No
<code>idx_producto_cat_disp_precio_cov</code>	producto(categoria_id, disponible, precio DESC) INCLUDE (id, nombre, stock)	14,185 → 9,311 ms (1,52x)	Evaluado, no aplicado	No (comentado)

Por qué un índice puede ser ignorado. Los casos descartados ilustran los motivos que enumera la guía de la unidad: baja selectividad (100 % y 50 % de las filas participan, así que no hay filas irrelevantes que evitar), estadísticas desactualizadas (falta de ANALYZE), tabla pequeña, discordancia de tipos y regla del prefijo más a la izquierda. El caso de `idx_producto_nombre_lower_vigente` mostró además un motivo técnico distinto: el primer intento, sin `text_pattern_ops`, siguió usando Seq Scan, porque en un locale distinto de «C» un B-Tree con la clase de operador por defecto no soporta LIKE de prefijo. Se corrigió y se volvió a medir en lugar de abandonar la idea.

Costo de escritura

Se midieron 500 INSERT en `detalle_pedido` (dentro de BEGIN...ROLLBACK) sin y con el índice descartado `idx_detalle_pedido_producto_id`: 272 ms sin el índice y 240 ms con él. Con ese volumen y una sola corrida, la variación del sistema pesa más que el costo real de mantener un índice, por lo que **la medición no permite concluir** nada sobre el costo de escritura. Se documenta tal como salió; la decisión de descartar ese índice se sostiene en que no mejoró la lectura.

6. Unidad 3 — Índices, vistas y objetos programables

6.1 Índices del esquema final (objetivo 4)

schema.sql deja seis índices creados a mano (además de los que crean automáticamente las PK y UNIQUE) y **objects.sql** suma el índice único de la vista materializada:

Índice	Tipo	Soporta
<code>idx_producto_categoria_id</code>	B-Tree simple	Listado de productos por categoría
<code>idx_pedido_usuario_id</code>	B-Tree simple	Historial de pedidos por usuario
<code>idx_producto_nombre_vigente</code>	B-Tree parcial (eliminado = FALSE)	Búsqueda por nombre exacto en el catálogo vigente
<code>idx_pedido_fecha_estado_vigente</code>	B-Tree compuesto parcial	Pedidos por rango de fechas y estado
<code>idx_pedido_usuario_total_estado</code>	B-Tree compuesto parcial (estado IN CONFIRMADO, TERMINADO)	Ranking y agregación de gasto por usuario (Index Only Scan)
<code>idx_producto_nombre_lower_vigente</code>	Índice de expresión con <code>text_pattern_ops</code> , parcial	Búsqueda por prefijo sin distinguir mayúsculas
<code>idx_mv_facturacion_cat_mes_pk</code>	UNIQUE (categoria_id, mes)	Requisito de REFRESH ... CONCURRENTLY en la vista materializada

6.2 Vistas (objetivo 6)

Cuatro vistas de reporte filtran por borrado lógico y una quinta protege datos sensibles:

Vista	Qué expone	Tablas base y filtro
<code>v_categorias_vigentes</code>	Categorías activas	<code>categoria · eliminado = FALSE</code>
<code>v_productos_vigentes</code>	Catálogo activo con el nombre de su categoría	<code>producto ⋈ categoria · ambos eliminado = FALSE</code>
<code>v_pedidos_resumen</code>	Cabecera de pedidos con el nombre completo del usuario	<code>pedido ⋈ usuario · pedido.eliminado = FALSE</code>
<code>v_pedido_detalle</code>	Líneas de cada pedido con el nombre del producto	<code>detalle_pedido ⋈ producto · detalle.eliminado = FALSE</code>
<code>v_usuario_seguro</code> (TP5)	Datos de usuario sin la columna contraseña ; pensada para dar SELECT a reportes sin exponer la tabla base	usuario (8 de 9 columnas; incluye eliminado como columna)

```
CREATE OR REPLACE VIEW v_usuario_seguro AS
SELECT id, nombre, apellido, mail, celular, rol, eliminado, created_at FROM usuario;
```

Verificación de equivalencia. Cada vista se comparó contra su consulta manual equivalente:

Vista	Filas de la vista	Consulta manual equivalente	Filas	Conclusión
<code>v_usuario_seguro</code>	20.006	<code>SELECT COUNT(*) FROM usuario</code>	20.006	Coincidencia exacta (no filtra eliminado; solo oculta la contraseña)
<code>v_productos_vigentes</code>	50.015	<code>... FROM producto WHERE eliminado = false</code>	50.015	Coincidencia exacta
<code>v_pedidos_resumen</code>	200.005	<code>SELECT COUNT(*) FROM pedido</code>	200.006	Equivalente: la diferencia son los pedidos con eliminado = TRUE
<code>v_pedido_detalle</code>	400.011	<code>SELECT COUNT(*) FROM detalle_pedido</code>	400.013	Equivalente: la diferencia son los detalles con eliminado = TRUE

Aclaración. Las diferencias de 1 pedido y 2 detalles entre vista y tabla se deben exclusivamente al borrado lógico: `v_pedido_detalle` filtra por `detalle_pedido.eliminado` y no por la vigencia del producto. En la verificación adicional de este informe, la diferencia coincidió exactamente con el conteo de filas con `eliminado = TRUE`. Los conteos de la tabla anterior son los de la corrida del grupo; los volúmenes finales de la Captura 2 son posteriores a las altas de demostración de `queries.sql`.

6.3 Función, triggers y procedimiento (objetivo 6)

Función `calcular_total_pedido(p_pedido_id)`: SQL puro (no necesita lógica de control) marcado *STABLE*, que suma los subtotales vigentes de un pedido. La usa el trigger de recálculo.

```
CREATE OR REPLACE FUNCTION calcular_total_pedido(p_pedido_id BIGINT) RETURNS NUMERIC(12,2) AS $$
    SELECT COALESCE(SUM(subtotal), 0) FROM detalle_pedido WHERE pedido_id = p_pedido_id AND eliminado = FALSE;
$$ LANGUAGE sql STABLE;
```

Triggers de subtotal y total. `trg_subtotal` (BEFORE INSERT/UPDATE, FOR EACH ROW) modifica *NEW* antes de escribir: si no llega *precio_unitario* toma el precio vigente del producto (lo «congela») y siempre calcula el subtotal. `trg_total_ins` y `trg_total_upd` (AFTER, FOR EACH STATEMENT) usan una *transition table* (*afectados*) para recalcular el total una sola vez por pedido y por sentencia. Se necesitan dos triggers porque PostgreSQL no admite una transition table para más de un evento en el mismo CREATE TRIGGER.

```
CREATE TRIGGER trg_subtotal BEFORE INSERT OR UPDATE ON detalle_pedido
    FOR EACH ROW EXECUTE FUNCTION fn_set_subtotal();
CREATE TRIGGER trg_total_ins AFTER INSERT ON detalle_pedido
    REFERENCING NEW TABLE AS afectados FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
CREATE TRIGGER trg_total_upd AFTER UPDATE ON detalle_pedido
    REFERENCING NEW TABLE AS afectados FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
```

Trigger de transición de estado (TP2). Impide que un pedido CONFIRMADO vuelva a PENDIENTE; cualquier otra transición se permite.

```
CREATE OR REPLACE FUNCTION fn_validar_estado_pedido() RETURNS TRIGGER AS $$
BEGIN
    IF OLD.estado = 'CONFIRMADO' AND NEW.estado = 'PENDIENTE' THEN
        RAISE EXCEPTION 'No se permite cambiar el estado de un pedido de CONFIRMADO a PENDIENTE (Pedido ID: %)', OLD.id;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_validar_estado_pedido BEFORE UPDATE OF estado ON pedido
    FOR EACH ROW EXECUTE FUNCTION fn_validar_estado_pedido();
```

Procedimiento `sp_crear_pedido`: descrito y probado en §4.6 (bloqueo de filas, validaciones de negocio, descuento de stock y atomicidad).

6.4 Vista materializada (objetivo 6)

`mv_facturacion_categoria_mes` guarda la facturación por categoría y mes (cantidad de pedidos distintos y total facturado) solo de pedidos *CONFIRMADO* y no eliminados. Tiene un índice único sobre (*categoria_id*, *mes*), requisito para poder refrescarla con *CONCURRENTLY* sin bloquear las lecturas.

```
CREATE MATERIALIZED VIEW mv_facturacion_categoria_mes AS
SELECT c.id AS categoria_id, c.nombre AS categoria,
    DATE_TRUNC('month', p.fecha)::DATE AS mes,
    COUNT(DISTINCT p.id) AS total_pedidos,
    SUM(dp.cantidad * dp.precio_unitario) AS total_facturado
FROM categoria c
JOIN producto pr ON pr.categoria_id = c.id
JOIN detalle_pedido dp ON dp.producto_id = pr.id
JOIN pedido p ON p.id = dp.pedido_id
WHERE p.estado = 'CONFIRMADO' AND p.eliminado = FALSE
GROUP BY c.id, c.nombre, DATE_TRUNC('month', p.fecha)
ORDER BY mes DESC, total_facturado DESC
WITH DATA;
CREATE UNIQUE INDEX idx_mv_facturacion_cat_mes_pk ON mv_facturacion_categoria_mes (categoria_id, mes);
```

Medición (EXPLAIN ANALYZE)	Tiempo
Consulta original sobre las tablas base (4 JOIN + agregación)	164,285 ms
Consulta sobre la vista materializada	0,044 ms (~3.700x más rápido)

Estrategia de refresco. La vista **no se actualiza sola**: entre un refresco y el siguiente queda «congelada» en el estado del último `REFRESH MATERIALIZED VIEW CONCURRENTLY mv_facturacion_categoria_mes`; Se recomienda un refresco **diario** (tarea programada a medianoche) o semanal: es un reporte gerencial, no un panel operativo, así que aceptar hasta 24 horas de desfase es razonable (un pedido confirmado a las 10 h no suma al mes hasta el próximo refresco). Si se necesitara mayor frescura conviene subir la frecuencia a cada pocas horas, sin llegar a refrescar en cada escritura, porque eso anularía el beneficio. Además, como *objects.sql* corre antes de cargar datos, la vista nace vacía y debe refrescarse después de la carga (§2.2).

7. Borrado lógico (soft delete) y su impacto (objetivo 9)

7.1 Mecanismo

Las cinco tablas tienen la columna `eliminado` `BOOLEAN NOT NULL DEFAULT FALSE`. Una «baja» es un `UPDATE` que la pone en `TRUE`; no se usa `DELETE` físico, y la clave foránea de `detalle_pedido.pedido_id` declara `ON DELETE RESTRICT` como red de seguridad. Así, si una categoría, un producto o un usuario «desaparece», los pedidos históricos que lo referencian siguen íntegros y el historial de ventas no se pierde.

7.2 Impacto en las consultas

Toda lectura de datos vigentes debe filtrar `eliminado = FALSE`. Para no repetir el filtro (y los `JOIN`) en cada consulta, las cuatro vistas de reporte lo encapsulan (§6.2); las consultas analíticas de `queries.sql` lo incluyen en cada tabla que participa (por ejemplo `p.eliminado = FALSE` en el `JOIN` de pedidos del ranking). La baja lógica de un pedido se hace en una transacción que marca primero sus detalles y luego la cabecera (HU-PED-03 de `queries.sql`):

```
BEGIN;
  UPDATE detalle_pedido SET eliminado = TRUE WHERE pedido_id = 6;
  UPDATE pedido        SET eliminado = TRUE WHERE id = 6;
COMMIT;
```

Verificación sobre `data.sql` (PostgreSQL 16.15) antes y después de esa transacción:

Medición	Antes de la baja	Después de la baja
Pedido 6: eliminado / total	FALSE / 3.200,00	TRUE / 0,00
Filas en la vista <code>v_pedidos_resumen</code> (la tabla <code>pedido</code> no cambia: 7)	6	5
Filas en la vista <code>v_pedido_detalle</code> (la tabla <code>detalle_pedido</code> no cambia: 13)	11	10

Consecuencia a tener presente: como el `UPDATE` de los detalles dispara `trg_total_upd` y `calcular_total_pedido` suma solo detalles vigentes, el **total del pedido dado de baja pasa a 0,00**. La información sigue en las filas (con `eliminado = TRUE`), pero `pedido.total` deja de reflejar el monto original; cualquier reporte que quiera usarlo debe filtrar por vigencia, como ya hacen las vistas y el ranking.

7.3 Impacto en los índices

Índices parciales. Cuatro de los seis índices son parciales con `WHERE eliminado = FALSE`: indexan solo filas vigentes, por lo que son más chicos y más baratos de mantener. La contracara es que el optimizador **solo puede usarlos si la consulta incluye ese mismo predicado**. Se comprobó con el índice de búsqueda por nombre sobre 50.017 productos (verificación adicional de este informe):

Consulta	Plan	Tiempo
<code>... WHERE lower(nombre) LIKE 'hamb%' AND eliminado = FALSE</code>	Bitmap Index Scan sobre <code>idx_producto_nombre_lower_vigente</code>	0,053 ms
<code>... WHERE lower(nombre) LIKE 'hamb%' (sin el predicado)</code>	Seq Scan; descarta 50.015 filas	14,760 ms

La misma consulta es ~280 veces más lenta si se olvida el filtro de borrado lógico. Esto explica que el índice `covering` de la competencia (§5.4-D) **no sea parcial**: la consulta entregada por la cátedra no incluía `eliminado = FALSE`, y la IA lo señaló como una omisión respecto de la convención del proyecto, que se documentó pero no se corrigió por alterar la consigna.

Restricciones UNIQUE. Un registro dado de baja **sigue ocupando su valor único**: al marcar como eliminado a un usuario, no se puede registrar otro con el mismo mail (`duplicate key value violates unique constraint «usuario_mail_key»` — comprobado), y lo mismo ocurre con `categoria.nombre`. Es una decisión razonable (evita reasignar un mail a otra persona), pero si se quisiera reutilizar valores habría que reemplazar el `UNIQUE` por un índice único parcial `WHERE eliminado = FALSE`. No se aplicó.

Vista materializada. Filtra `p.eliminado = FALSE` (y solo pedidos CONFIRMADO). Como la baja lógica del sistema se aplica al pedido junto con todos sus detalles, no hay diferencia hoy; si en el futuro se permitiera dar de baja detalles de forma individual dentro de un pedido vigente, convendría agregar `dp.eliminado = FALSE` a la definición.

8. Uso de herramientas de IA

El grupo trabajó con un flujo de especificaciones (*specs* en **Kiro**) y generación de SQL con **OpenCode**, y además usó **Gemini**, **Nemotron** y **Claude** (Anthropic). Cada integrante registró sus interacciones en una Declaración de Uso de IA (DUIA). El criterio común fue no aceptar ninguna propuesta sin contrastarla con el esquema real (*\d tabla*, lectura línea a línea de *objects.sql*), con el plan de ejecución medido o con el comportamiento del motor.

Parte	Quién	Herramientas	Para qué	Aceptado / descartado y por qué
Concurrencia (TP2)	Equipo	Asistente de IA	Explicar cada escenario (lectura fantasma, deadlock, espera de fila) y proponer cómo evitarlo	Aceptado tras verificar en el motor: con REPEATABLE READ el COUNT se mantuvo en 6; el orden consistente de bloqueo evitó el deadlock; la espera de fila se observó tal cual.
Competencia de optimización (TP3)	Andrés	IA (prompt con consulta y plan real)	Proponer índices o reescrituras justificando cada uno contra un nodo del plan	Aceptado: índice covering (Propuesta 2, 1,52x). No medidos: índice compuesto simple y parcial. Descartado: agregar <i>eliminado = FALSE</i> a la consulta, por alterar la consulta de la cátedra. Se corrigió a mano la columna inexistente <i>activo</i> → <i>disponible</i> .
Plan de indexado (TP5, Parte A)	Mariano	Kiro, OpenCode	Validar descartes de las consultas 1 y 2 y proponer el índice de la consulta 3	Aceptado: descartes confirmados con las mediciones; índice con <i>text_pattern_ops</i> propuesto desde el primer intento con la explicación del locale. Se reportó el costo de escritura tal como salió (no concluyente).
Vistas (TP5, Parte B)	Facundo	Kiro, OpenCode, Gemini	Redactar specs de las vistas, generar <i>v_usuario_seguro</i> y proponer el formato del reporte de equivalencias	Corregido: las 3 specs de Kiro usaban nombres de columna genéricos que no coincidían con <i>objects.sql</i> . Aceptado sin cambios: <i>v_usuario_seguro</i> (verificada contra <i>ld usuario</i>). Los conteos se ejecutaron a mano.
Vista materializada (TP5, Parte C)	Andrés	Kiro, OpenCode, Nemotron, Gemini	Especificar el reporte y generar la vista con su índice único	Corregido: la propuesta ubicaba <i>WITH DATA</i> al inicio; en PostgreSQL va al final, después del SELECT. Se ajustó antes de ejecutarla.
Redacción de este informe	Equipo (asistencia de Claude)	Claude	Armaz el informe a partir del repositorio y de la documentación del grupo; verificar en PostgreSQL 16 la reproducción de los scripts, las reglas de negocio y el borrado lógico	Aportes: corrección de la explicación de las diferencias de filas en las vistas (borrado lógico y no vigencia de producto); corrección de la notación PRODUCTO-DETALLE_PEDIDO en el ER; documentación honesta del índice covering como no aplicado y de las limitaciones (§9). El texto se revisa contra los archivos del repositorio.

9. Dificultades, decisiones y limitaciones conocidas

9.1 Dificultades y soluciones

Dificultad	Solución
Error tipográfico en el DDL de la cátedra: <i>BIGGINT</i> no existe en PostgreSQL	Corregido a <i>BIGINT</i> al ejecutar el script
Un comentario sin el prefijo <i>--</i> en <i>transacciones.sql</i> rompía la ejecución completa del script	Se corrigió y se recomienda ejecutar los escenarios de concurrencia por bloques, porque requieren intervención manual entre dos sesiones
pgAdmin solo muestra en «Data Output» el resultado de la última sentencia cuando se ejecutan varias juntas	Se ejecutaron de a una o dos sentencias, dejando el SELECT de verificación al final, para capturar cada estado
Simular dos operadores concurrentes en una sola máquina	Dos pestañas de Query Tool sobre la misma base, coordinando a mano el orden (A sin COMMIT mientras corre B)
Errores en <i>carga_masiva.sql</i> (fechas fuera de rango, ids de pedido asumidos mal)	Se corrigió el script, se recreó la copia de trabajo y se volvieron a medir todos los índices
El índice de <i>lower(nombre)</i> sin <i>text_pattern_ops</i> no era usado	Se agregó la clase de operador y se volvió a medir (50 ms → 0,22 ms)

Dificultad	Solución
La consulta de la competencia usaba la columna <i>activo</i> , que no existe	Se acordó como error de tipeo y se adaptó a <i>disponible</i>
La vista materializada nace vacía si <i>objects.sql</i> corre antes de los datos	Ejecutar el REFRESH después de la carga (§2.2 y §6.4)

9.2 Limitaciones y mejoras futuras

- **Índice covering** *idx_producto_cat_disp_precio_cov*: evaluado y no aplicado; queda comentado en *schema.sql* (§5.4-D).
- **UNIQUE y borrado lógico**: los valores únicos de registros eliminados quedan reservados; una alternativa es un índice único parcial (§7.3).
- **Vista materializada**: podría filtrar también *detalle_pedido.eliminado* si se admiten bajas individuales de detalles (§7.3).
- **Total de pedidos dados de baja**: pasa a 0,00 por el recálculo automático (§7.2); si se necesitara conservar el monto histórico habría que decidir otra política.
- **Costo de escritura de los índices**: la medición de 500 INSERT no fue concluyente; una medición robusta requeriría más filas y varias corridas promediadas (§5.5).
- **Índice de búsqueda por categoría con pocos datos**: con 5.000 filas el índice *idx_producto_categoria_id* redujo el costo estimado pero no el tiempo real; su beneficio se espera con el volumen productivo (§5.5).
- **Desempate del ranking**: el orden determinista por *u.id* impide que haya empates reales en el puesto (§5.2).
- **transacciones.sql**: en el Escenario 3, la línea *SET TRANSACTION ISOLATION LEVEL SERIALIZABLE* debe ejecutarse dentro de la transacción, inmediatamente después de BEGIN; fuera de ella PostgreSQL la ignora con una advertencia.

10. Conclusiones

La entrega cubre los nueve objetivos de la consigna con evidencia ejecutable: un esquema de cinco tablas normalizado con una denormalización controlada y documentada; reglas de negocio garantizadas por el motor (CHECK, UNIQUE, triggers y un procedimiento transaccional); siete escenarios que demuestran atomicidad, aislamiento y control de concurrencia; consultas con JOIN, agregación, subconsultas y funciones de ventana verificadas por equivalencia; y una optimización guiada por mediciones, que aceptó los índices que mejoraron el plan (hasta ~226x) y descartó con fundamento los que no. El borrado lógico atraviesa todas las capas (vistas, consultas, índices parciales y restricciones UNIQUE), y sus efectos secundarios se documentaron en lugar de ocultarse. El proyecto seguirá ampliándose en las unidades posteriores.

11. Bibliografía, repositorio y video

- PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/index.html> — tipos de datos, CREATE TABLE, restricciones, ENUM e IDENTITY.
- PostgreSQL Global Development Group. *Chapter 39. Triggers*. <https://www.postgresql.org/docs/16/triggers.html> — diseño de trg_subtotal, trg_total_ins/upd y transition tables.
- PostgreSQL Global Development Group. *PL/pgSQL — SQL Procedural Language*. <https://www.postgresql.org/docs/16/plpgsql.html> — sintaxis de sp_crear_pedido, JSONB y RAISE EXCEPTION.
- PostgreSQL Global Development Group. *Chapter 13. Concurrency Control*. <https://www.postgresql.org/docs/16/mvcc.html> — niveles de aislamiento y bloqueos.
- PostgreSQL Global Development Group. *13.2. Transaction Isolation*. <https://www.postgresql.org/docs/16/transaction-iso.html> — lectura no repetible y fantasma.
- PostgreSQL Global Development Group. *Explicit Locking*. <https://www.postgresql.org/docs/16/explicit-locking.html> — bloqueos de fila y deadlocks.
- PostgreSQL Global Development Group. *Chapter 11. Indexes*. <https://www.postgresql.org/docs/16/indexes.html> — índices parciales, de expresión, covering y clases de operador.

- PostgreSQL Global Development Group. *Materialized Views*. <https://www.postgresql.org/docs/16/rules-materializedviews.html> — REFRESH y CONCURRENTLY.
- PostgreSQL Global Development Group. *Using EXPLAIN*. <https://www.postgresql.org/docs/16/using-explain.html> — lectura de planes (Seq Scan, Bitmap Index Scan, Index Scan).
- Elmasri, R. y Navathe, S. B. (2017). *Fundamentals of Database Systems* (7.ª ed.). Pearson — modelo relacional, normalización y dependencias funcionales.
- Date, C. J. (2004). *An Introduction to Database Systems* (8.ª ed.). Addison-Wesley — propiedades ACID y claves candidatas.
- Apuntes, guías académicas y material de cátedra de Base de Datos 2 — UTN Facultad Regional Mendoza (Tecnatura Universitaria en Programación).

Recurso	Enlace
Repositorio del proyecto	https://github.com/marianoemir/TPI-BD-Food-Store
Video	https://youtu.be/GBiR7vITjdQ

Anexo A — Capturas de las mediciones de los índices base (TP1)

Planes de ejecución con **5.000 filas** en *producto* y 2.000 en *pedido* (base del TP1), sin y con cada índice. Los valores de costo y tiempo se resumen en \$5.5.

Query Query History

```

1  --Índice 1 — categoria_id en producto:*
2
3  DROP INDEX idx_producto_categoria_id;
4  EXPLAIN ANALYZE SELECT * FROM producto WHERE categoria_id = 1;
5  -- CAPTURA 1 (sin índice, debería decir "Seq Scan")
6

```

Data Output Messages Notifications

Showing rows: 1 to 8

QUERY PLAN
text

1	Seq Scan on producto (cost=0.00..124.50 rows=250 width=1081) (actual time=0.059..1.805 rows=250.00 loops=1)
2	Filter: (categoria_id = 1)
3	Rows Removed by Filter: 4750
4	Buffers: shared hit=62
5	Planning:
6	Buffers: shared hit=72 dirtied=2
7	Planning Time: 3.679 ms
8	Execution Time: 1.857 ms

Query Query History

```

6
7  CREATE INDEX idx_producto_categoria_id ON producto(categoria_id);
8  EXPLAIN ANALYZE SELECT * FROM producto WHERE categoria_id = 1;
9  -- CAPTURA 2 (con índice, debería decir "Index Scan")

```

Data Output Messages Notifications

Showing rows: 1 to 12

QUERY PLAN
text

1	Bitmap Heap Scan on producto (cost=6.22..71.34 rows=250 width=1081) (actual time=0.618..0.768 rows=250.00 loops=1)
2	Recheck Cond: (categoria_id = 1)
3	Heap Blocks: exact=62
4	Buffers: shared hit=65 read=2
5	-- Bitmap Index Scan on idx_producto_categoria_id (cost=0.00..6.16 rows=250 width=0) (actual time=0.577..0.578 rows=...
6	Index Cond: (categoria_id = 1)
7	Index Searches: 1
8	Buffers: shared hit=3 read=2
9	Planning:
10	Buffers: shared hit=9 read=1
11	Planning Time: 49.895 ms
12	Execution Time: 4.604 ms

Índice 1 — *idx_producto_categoria_id*: sin índice (Seq Scan) y con índice (Bitmap Heap Scan).

Query Query History

```

12  --Índice 2 — usuario_id en pedido:*
13
14  DROP INDEX idx_pedido_usuario_id;
15  EXPLAIN ANALYZE SELECT * FROM pedido WHERE usuario_id = 1;
16  -- CAPTURA 3

```

Data Output Messages Notifications

Showing rows: 1 to 8

QUERY PLAN
text

1	Seq Scan on pedido (cost=0.00..44.00 rows=4 width=40) (actual time=0.217..0.556 rows=4.00 loops=1)
2	Filter: (usuario_id = 1)
3	Rows Removed by Filter: 1996
4	Buffers: shared hit=19
5	Planning:
6	Buffers: shared hit=40 dirtied=1
7	Planning Time: 4.009 ms
8	Execution Time: 0.665 ms

Query Query History

```

17
18 CREATE INDEX idx_pedido_usuario_id ON pedido(usuario_id);
19 EXPLAIN ANALYZE SELECT * FROM pedido WHERE usuario_id = 1;
20 -- CAPTURA 4

```

Data Output Messages Notifications

Showing rows: 1 to 12 SQL

QUERY PLAN
text
1 Bitmap Heap Scan on pedido (cost=4.31..14.85 rows=4 width=40) (actual time=0.120..0.126 rows=4.00 loops=1)
2 Recheck Cond: (usuario_id = 1)
3 Heap Blocks: exact=4
4 Buffers: shared hit=4 read=2
5 -> Bitmap Index Scan on idx_pedido_usuario_id (cost=0.00..4.31 rows=4 width=0) (actual time=0.091..0.091 rows=4.00 lo...
6 Index Cond: (usuario_id = 1)
7 Index Searches: 1
8 Buffers: shared read=2
9 Planning:
10 Buffers: shared hit=5 read=1
11 Planning Time: 2.602 ms
12 Execution Time: 0.160 ms

Índice 2 — *idx_pedido_usuario_id*: sin índice (Seq Scan) y con índice (Bitmap Heap Scan).

Query Query History

```

23 --Índice 3 — nombre en producto (parcial):*
24
25 DROP INDEX idx_producto_nombre_vigente;
26 EXPLAIN ANALYZE SELECT * FROM producto WHERE nombre = 'Producto 100' AND eliminado = FALSE;
27 -- CAPTURA 5
28

```

Data Output Messages Notifications

Showing rows: 1 to 8 SQL

QUERY PLAN
text
1 Seq Scan on producto (cost=0.00..124.50 rows=1 width=1081) (actual time=0.064..1.521 rows=1.00 loops=1)
2 Filter: ((NOT eliminado) AND ((nombre)::text = 'Producto 100'::text))
3 Rows Removed by Filter: 4999
4 Buffers: shared hit=62
5 Planning:
6 Buffers: shared hit=17
7 Planning Time: 3.062 ms
8 Execution Time: 1.552 ms

Query Query History

```

30
31 CREATE INDEX idx_producto_nombre_vigente ON producto(nombre) WHERE eliminado = FALSE;
32 EXPLAIN ANALYZE SELECT * FROM producto WHERE nombre = 'Producto 100' AND eliminado = FALSE;
33 -- CAPTURA 6
34

```

Data Output Messages Notifications

Showing rows: 1 to 8 SQL

QUERY PLAN
text
1 Index Scan using idx_producto_nombre_vigente on producto (cost=0.28..8.30 rows=1 width=1081) (actual time=0.086..0.087 rows=1.00 lo...
2 Index Cond: ((nombre)::text = 'Producto 100'::text)
3 Index Searches: 1
4 Buffers: shared hit=1 read=2
5 Planning:
6 Buffers: shared hit=10 read=1
7 Planning Time: 2.692 ms
8 Execution Time: 0.111 ms

Índice 3 — *idx_producto_nombre_vigente* (parcial): sin índice (Seq Scan) y con índice (Index Scan).

Anexo B — Script completo: schema.sql

Versión consolidada (schema.sql). Ver §4.4 «DDL completo» y §6.1 «Índices del esquema final».

```

1  -- =====
2  -- FOOD STORE - schema.sql (CONSOLIDADO – Entrega 1 del TPI)
3  -- Tipos, tablas, restricciones e índices
4  --
5  -- Este archivo fusiona:
6  --   - schema.sql original (TP1)
7  --   - los 2 CHECK agregados en el TP2 (mail, fecha)
8  --   - los 3 índices agregados en TP3/TP4 (indices_semana3.sql) y TP5 (indices.sql)
9  -- =====
10
11 -- Tipos enumerados
12 CREATE TYPE rol AS ENUM ('ADMIN','USUARIO');
13 CREATE TYPE estado_pedido AS ENUM ('PENDIENTE','CONFIRMADO','TERMINADO','CANCELADO');
14 CREATE TYPE forma_pago AS ENUM ('TARJETA','TRANSFERENCIA','EFECTIVO');
15
16 -- =====
17 -- CATEGORIA
18 -- =====
19 CREATE TABLE categoria (
20   id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, -- corregido: BIGGINT -> BIGIN
21   T
22   nombre      VARCHAR(80) NOT NULL UNIQUE,
23   descripcion  VARCHAR(255),
24   eliminado    BOOLEAN NOT NULL DEFAULT FALSE,
25   created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
26 );
27
28 -- =====
29 -- PRODUCTO
30 -- =====
31 CREATE TABLE producto (
32   id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
33   nombre      VARCHAR(120) NOT NULL,
34   precio      NUMERIC(10,2) NOT NULL CHECK (precio >= 0),
35   descripcion  VARCHAR(255),
36   stock       INTEGER NOT NULL DEFAULT 0 CHECK (stock >= 0),
37   imagen       VARCHAR(255),
38   disponible   BOOLEAN NOT NULL DEFAULT TRUE,
39   categoria_id BIGINT NOT NULL REFERENCES categoria(id),
40   eliminado    BOOLEAN NOT NULL DEFAULT FALSE,
41   created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
42 );
43
44 -- =====
45 -- USUARIO
46 -- =====
47 CREATE TABLE usuario (
48   id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
49   nombre      VARCHAR(80) NOT NULL,
50   apellido     VARCHAR(80) NOT NULL,
51   mail         VARCHAR(120) NOT NULL UNIQUE,
52   celular      VARCHAR(30),
53   contrasena   VARCHAR(255) NOT NULL,
54   rol          rol NOT NULL DEFAULT 'USUARIO',
55   eliminado    BOOLEAN NOT NULL DEFAULT FALSE,
56   created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
57   -- Agregado en TP2 (Semana 2 – Concurrencia e IA)
58   CONSTRAINT chk_usuario_mail_formato CHECK (
59     mail ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
60   );
61
62 -- =====
63 -- PEDIDO
64 -- =====
65 CREATE TABLE pedido (
66   id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
67   -- CHECK fecha <= CURRENT_DATE agregado en TP2
68   fecha       DATE NOT NULL DEFAULT CURRENT_DATE CHECK (fecha <= CURRENT_DATE),
69   estado      estado_pedido NOT NULL DEFAULT 'PENDIENTE',
70   total       NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (total >= 0),

```

Anexo B — schema.sql (continuación)

```

71     forma_pago     forma_pago NOT NULL,
72     usuario_id     BIGINT NOT NULL REFERENCES usuario(id),
73     eliminado       BOOLEAN NOT NULL DEFAULT FALSE,
74     created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
75 );
76
77 -- =====
78 -- DETALLE_PEDIDO
79 -- =====
80 CREATE TABLE detalle_pedido (
81     id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
82     cantidad        INTEGER NOT NULL CHECK (cantidad > 0),
83     precio_unitario NUMERIC(10,2) NOT NULL CHECK (precio_unitario >= 0),
84     subtotal        NUMERIC(12,2) NOT NULL CHECK (subtotal >= 0),
85     pedido_id       BIGINT NOT NULL REFERENCES pedido(id) ON DELETE RESTRICT,
86     producto_id     BIGINT NOT NULL REFERENCES producto(id),
87     eliminado       BOOLEAN NOT NULL DEFAULT FALSE,
88     created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
89     UNIQUE (pedido_id, producto_id)
90 );
91
92 -- =====
93 -- ÍNDICES – originales (TP1)
94 -- =====
95
96 -- Soporta el listado de productos por categoría
97 CREATE INDEX idx_producto_categoria_id ON producto(categoria_id);
98
99 -- Soporta el historial de pedidos por usuario
100 CREATE INDEX idx_pedido_usuario_id ON pedido(usuario_id);
101
102 -- Índice parcial: solo productos no eliminados, para búsquedas por nombre
103 CREATE INDEX idx_producto_nombre_vigente ON producto(nombre) WHERE eliminado = FALSE;
104
105 -- =====
106 -- ÍNDICES – agregados en TP3 (medidos y aceptados en indices_semana3.sql)
107 -- =====
108
109 -- Historial de pedidos por rango de fechas y estado.
110 -- Medido: Seq Scan -> Bitmap Heap Scan, ~2.1x más rápido (27.9ms -> 13.3ms)
111 CREATE INDEX idx_pedido_fecha_estado_vigente
112 ON pedido(fecha, estado)
113 WHERE eliminado = FALSE;
114
115 -- Ranking / agregación de gasto por usuario.
116 -- Medido: Seq Scan -> Index Only Scan (Heap Fetches: 0), ~1.7x más rápido (88.9ms -> 52.3ms)
117 CREATE INDEX idx_pedido_usuario_total_estado
118 ON pedido(usuario_id, total)
119 WHERE eliminado = FALSE AND estado IN ('CONFIRMADO', 'TERMINADO');
120
121 -- Nota: idx_producto_categoria_precio_vigente (evaluado en el TP3) se descartó
122 -- a propósito – el optimizador lo ignoró y no dio mejora medible.
123
124 -- =====
125 -- ÍNDICES – agregados en TP5 / Unidad 3 (indices.sql)
126 -- =====
127
128 -- Búsqueda de productos por nombre, case-insensitive (LIKE 'prefijo%').
129 -- Requiere text_pattern_ops para que el índice soporte ese operador.
130 -- Medido: ~226x más rápido (50.2ms -> 0.22ms), Seq Scan -> Bitmap Heap Scan
131 CREATE INDEX idx_producto_nombre_lower_vigente
132 ON producto (lower(nombre) text_pattern_ops)
133 WHERE eliminado = FALSE;
134
135 -- PENDIENTE DE DECISIÓN DEL GRUPO:
136 -- idx_producto_cat_disp_precio_cov (índice covering ganador de la
137 -- "competencia" del TP3) no está incluido acá todavía. Si lo van a usar en
138 -- alguna consulta del informe final, sumarlo con:
139 -- CREATE INDEX idx_producto_cat_disp_precio_cov
140 -- ON producto (categoria_id, disponible, precio DESC)
141 -- INCLUDE (id, nombre, stock);
142
143 -- Actualizar estadísticas después de crear los índices nuevos
144 ANALYZE producto;
145 ANALYZE pedido;

```

Anexo C — Script completo: objects.sql

Versión consolidada (objects.sql). Ver §6.2 a §6.4 «Vistas, función, triggers, procedimiento y vista materializada».

```

1  -- =====
2  -- FOOD STORE - objects.sql (CONSOLIDADO – Entrega 1 del TPI)
3  -- Vistas, función de total, triggers, procedimiento de pedido,
4  -- vista de seguridad y vista materializada
5  --
6  -- Este archivo fusiona:
7  --   - objects.sql original (TP1): 4 vistas, función, triggers de subtotal/total, sp_crear_pedido
8  --   - el trigger de validación de estado agregado en TP2
9  --   - la vista v_usuario_seguro agregada en TP5 (views.sql)
10 --   - la vista materializada agregada en TP5 (materializadas.sql)
11 --
12 -- Requiere haber corrido schema.sql antes.
13 -- =====
14
15 -- =====
16 -- 1) VISTAS OBLIGATORIAS (TP1)
17 -- =====
18
19 CREATE VIEW v_categorias_vigentes AS
20 SELECT id, nombre, descripcion
21 FROM categoria
22 WHERE eliminado = FALSE;
23
24 CREATE VIEW v_productos_vigentes AS
25 SELECT p.id, p.nombre, p.precio, p.stock,
26        c.nombre AS categoria
27 FROM producto p
28 JOIN categoria c ON c.id = p.categoria_id
29 WHERE p.eliminado = FALSE AND c.eliminado = FALSE;
30
31 CREATE VIEW v_pedidos_resumen AS
32 SELECT ped.id,
33        u.nombre || ' ' || u.apellido AS usuario,
34        ped.fecha, ped.estado, ped.forma_pago, ped.total
35 FROM pedido ped
36 JOIN usuario u ON u.id = ped.usuario_id
37 WHERE ped.eliminado = FALSE;
38
39 CREATE VIEW v_pedido_detalle AS
40 SELECT dp.pedido_id,
41        pr.nombre AS producto,
42        dp.cantidad, dp.precio_unitario, dp.subtotal
43 FROM detalle_pedido dp
44 JOIN producto pr ON pr.id = dp.producto_id
45 WHERE dp.eliminado = FALSE;
46
47 -- =====
48 -- 1b) VISTA DE SEGURIDAD (agregada en TP5 – views.sql)
49 -- Expone datos de usuario ocultando la contraseña.
50 -- =====
51
52 CREATE OR REPLACE VIEW v_usuario_seguro AS
53 SELECT
54     id,
55     nombre,
56     apellido,
57     mail,
58     celular,
59     rol,
60     eliminado,
61     created_at
62 FROM usuario;
63
64 COMMENT ON VIEW v_usuario_seguro IS 'Vista segura de usuarios que excluye la columna sensible de
65     contraseña para reportes generales.';
66
67 -- =====
68 -- 2) FUNCIÓN DE CÁLCULO DE TOTAL (TP1)
69 -- =====

```

Anexo C — objects.sql (continuación)

```

70 CREATE OR REPLACE FUNCTION calcular_total_pedido(p_pedido_id BIGINT)
71 RETURNS NUMERIC(12,2) AS $$
72     SELECT COALESCE(SUM(subtotal), 0)
73     FROM detalle_pedido
74     WHERE pedido_id = p_pedido_id AND eliminado = FALSE;
75 $$ LANGUAGE sql STABLE;
76
77 -- =====
78 -- 3) TRIGGERS: subtotal y total automáticos (TP1)
79 -- =====
80
81 -- Congela el precio del producto y calcula el subtotal
82 -- ANTES de insertar o actualizar una línea de detalle.
83 CREATE OR REPLACE FUNCTION fn_set_subtotal()
84 RETURNS TRIGGER AS $$
85 BEGIN
86     IF NEW.precio_unitario IS NULL THEN
87         SELECT precio INTO NEW.precio_unitario
88         FROM producto WHERE id = NEW.producto_id;
89     END IF;
90
91     NEW.subtotal := NEW.cantidad * NEW.precio_unitario;
92     RETURN NEW;
93 END;
94 $$ LANGUAGE plpgsql;
95
96 CREATE TRIGGER trg_subtotal
97 BEFORE INSERT OR UPDATE ON detalle_pedido
98 FOR EACH ROW EXECUTE FUNCTION fn_set_subtotal();
99
100 -- Recalcula pedido.total sumando los subtotales vigentes,
101 -- DESPUÉS de que cambian los detalles (una sola pasada por sentencia).
102 CREATE OR REPLACE FUNCTION fn_recalcular_total()
103 RETURNS TRIGGER AS $$
104 BEGIN
105     UPDATE pedido p
106     SET total = calcular_total_pedido(p.id)
107     WHERE p.id IN (SELECT pedido_id FROM afectados);
108     RETURN NULL;
109 END;
110 $$ LANGUAGE plpgsql;
111
112 -- Un trigger por evento (INSERT y UPDATE por separado):
113 -- las transition tables no admiten declarar varios eventos juntos.
114 CREATE TRIGGER trg_total_ins
115 AFTER INSERT ON detalle_pedido
116 REFERENCING NEW TABLE AS afectados
117 FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
118
119 CREATE TRIGGER trg_total_upd
120 AFTER UPDATE ON detalle_pedido
121 REFERENCING NEW TABLE AS afectados
122 FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
123
124 -- =====
125 -- 3b) TRIGGER de transición de estado (agregado en TP2)
126 -- Valida que un pedido CONFIRMADO no pueda volver a estado PENDIENTE
127 -- =====
128
129 CREATE OR REPLACE FUNCTION fn_validar_estado_pedido()
130 RETURNS TRIGGER AS $$
131 BEGIN
132     IF OLD.estado = 'CONFIRMADO' AND NEW.estado = 'PENDIENTE' THEN
133         RAISE EXCEPTION 'No se permite cambiar el estado de un pedido de CONFIRMADO a PENDIENTE
(Pedido ID: %)', OLD.id;
134     END IF;
135     RETURN NEW;
136 END;
137 $$ LANGUAGE plpgsql;
138
139 CREATE TRIGGER trg_validar_estado_pedido
140 BEFORE UPDATE OF estado ON pedido
141 FOR EACH ROW EXECUTE FUNCTION fn_validar_estado_pedido();
142
143 -- =====
144 -- 4) PROCEDIMIENTO TRANSACCIONAL: alta de pedido con detalles (TP1)

```

Anexo C — objects.sql (continuación)

```

145 -- =====
146
147 CREATE OR REPLACE PROCEDURE sp_crear_pedido(
148     p_usuario_id BIGINT,
149     p_forma_pago forma_pago,
150     p_items JSONB -- [{"producto_id":1,"cantidad":2}, ...]
151 ) AS $$
152 DECLARE
153     v_pedido_id      BIGINT;
154     v_item            JSONB;
155     v_producto_id    BIGINT;
156     v_cantidad       INTEGER;
157     v_stock          INTEGER;
158     v_disponible     BOOLEAN;
159 BEGIN
160     -- El usuario debe existir y no estar eliminado
161     IF NOT EXISTS (SELECT 1 FROM usuario
162                    WHERE id = p_usuario_id AND eliminado = FALSE) THEN
163         RAISE EXCEPTION 'Usuario % inexistente o eliminado', p_usuario_id;
164     END IF;
165
166     INSERT INTO pedido(usuario_id, forma_pago)
167     VALUES (p_usuario_id, p_forma_pago)
168     RETURNING id INTO v_pedido_id;
169
170     FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
171         v_producto_id := (v_item->>'producto_id')::BIGINT;
172         v_cantidad     := (v_item->>'cantidad')::INTEGER;
173
174         -- Bloquea la fila del producto para evitar sobreventa concurrente
175         SELECT stock, disponible INTO v_stock, v_disponible
176         FROM producto WHERE id = v_producto_id AND eliminado = FALSE
177         FOR UPDATE;
178
179         IF NOT FOUND THEN
180             RAISE EXCEPTION 'Producto % inexistente o eliminado', v_producto_id;
181         END IF;
182
183         IF NOT v_disponible THEN
184             RAISE EXCEPTION 'Producto % no disponible', v_producto_id;
185         END IF;
186
187         IF v_stock < v_cantidad THEN
188             RAISE EXCEPTION 'Stock insuficiente (producto %): hay %, se piden %',
189                 v_producto_id, v_stock, v_cantidad;
190         END IF;
191
192         INSERT INTO detalle_pedido(pedido_id, producto_id, cantidad)
193         VALUES (v_pedido_id, v_producto_id, v_cantidad);
194
195         -- Descuenta stock dentro de la misma transacción
196         UPDATE producto SET stock = stock - v_cantidad WHERE id = v_producto_id;
197     END LOOP;
198
199     -- Si alguna inserción falla, toda la transacción se revierte (rollback).
200 END;
201 $$ LANGUAGE plpgsql;
202
203 -- =====
204 -- 5) VISTA MATERIALIZADA (agregada en TP5 – materializadas.sql)
205 -- Facturación por categoría y mes.
206 -- =====
207
208 CREATE MATERIALIZED VIEW mv_facturacion_categoria_mes AS
209 SELECT
210     c.id AS categoria_id,
211     c.nombre AS categoria,
212     DATE_TRUNC('month', p.fecha)::DATE AS mes,
213     COUNT(DISTINCT p.id) AS total_pedidos,
214     SUM(dp.cantidad * dp.precio_unitario) AS total_facturado
215 FROM categoria c
216 JOIN producto pr ON pr.categoria_id = c.id
217 JOIN detalle_pedido dp ON dp.producto_id = pr.id
218 JOIN pedido p ON p.id = dp.pedido_id
219 WHERE p.estado = 'CONFIRMADO' AND p.eliminado = FALSE
220 GROUP BY c.id, c.nombre, DATE_TRUNC('month', p.fecha)

```

Anexo C — objects.sql (continuación)

```
221 ORDER BY mes DESC, total_facturado DESC
222 WITH DATA;
223
224 -- Índice único: requisito para poder hacer REFRESH ... CONCURRENTLY
225 CREATE UNIQUE INDEX idx_mv_facturacion_cat_mes_pk
226 ON mv_facturacion_categoria_mes (categoria_id, mes);
227
228 -- Nota para el informe: esta vista materializada NO se actualiza sola.
229 -- Hay que documentar cuándo/cómo se refresca, por ejemplo:
230 -- REFRESH MATERIALIZED VIEW CONCURRENTLY mv_facturacion_categoria_mes;
```

Anexo D — Script completo: queries.sql

Versión consolidada (queries.sql). Ver §5.2 «Consultas: JOIN, agregación, subconsultas y funciones de ventana».

```

1  -- =====
2  -- FOOD STORE - queries.sql (Consolidado - Bloque B)
3  -- Proyecto Integrador - Base de Datos II (PostgreSQL 16+)
4  -- =====
5  -- Este script consolida la ejecución de consultas operativas (HUs),
6  -- agregaciones, subconsultas avanzadas, funciones de ventana y
7  -- pruebas de equivalencia con EXCEPT.
8  -- Correr DESPUÉS de: schema.sql -> objects.sql -> data.sql / carga_masiva.sql
9  -- =====
10
11
12 -- =====
13 -- SECCIÓN 1: HISTORIAS DE USUARIO (OPERATIVO Y DDL/DML)
14 -- =====
15 -- NOTA: los IDs usados en esta sección (categoria id=1, usuario id=3/5,
16 -- pedido id=6, producto id=1/10) asumen los datos cargados por data.sql
17 -- SIN modificaciones previas. Las bajas lógicas (HU-USR-03, HU-PED-03)
18 -- no son idempotentes: si esta sección ya se corrió una vez sobre una
19 -- base, volver a correrla puede no tener el mismo efecto (los registros
20 -- ya estarán con eliminado = TRUE). Para repetir la demo desde cero,
21 -- recargar data.sql antes de correr esta sección.
22 -- =====
23
24 -- -----
25 -- 1.1 ÉPICA: GESTIÓN DE CATEGORÍAS
26 -- -----
27
28 -- HU-CAT-01: Listar categorías vigentes (borrado lógico aplicado)
29 SELECT id, nombre, descripcion
30 FROM categoria
31 WHERE eliminado = FALSE
32 ORDER BY id;
33
34 -- HU-CAT-02: Crear categoría (Caso feliz - Evita duplicados en reejecuciones)
35 INSERT INTO categoria(nombre, descripcion)
36 SELECT 'Vegano', 'Opciones sin productos de origen animal'
37 WHERE NOT EXISTS (
38     SELECT 1 FROM categoria WHERE nombre = 'Vegano'
39 );
40
41 -- HU-CAT-03: Editar categoría
42 UPDATE categoria
43 SET nombre = 'Pizzas Artesanales', descripcion = 'Pizzas a la piedra, catálogo ampliado'
44 WHERE id = 1 AND eliminado = FALSE;
45
46 -- HU-CAT-04: Eliminar categoría (Baja lógica)
47 UPDATE categoria
48 SET eliminado = TRUE
49 WHERE nombre = 'Vegano' AND eliminado = FALSE;
50
51 -- Verificación en vista
52 SELECT * FROM v_categorias_vigentes WHERE nombre = 'Vegano';
53
54
55 -- -----
56 -- 1.2 ÉPICA: GESTIÓN DE PRODUCTOS
57 -- -----
58
59 -- HU-PROD-01: Listar productos vigentes con su categoría (JOIN explícito)
60 SELECT p.id, p.nombre, p.precio, p.stock, c.nombre AS categoria
61 FROM producto p
62 JOIN categoria c ON c.id = p.categoria_id
63 WHERE p.eliminado = FALSE
64 ORDER BY p.id;
65
66 -- HU-PROD-02: Crear producto validando categoría vigente (Evita duplicados)
67 INSERT INTO producto(nombre, descripcion, precio, stock, disponible, categoria_id)
68 SELECT 'Calzone', 'Pizza cerrada rellena', 4700.00, 8, TRUE, c.id
69 FROM categoria c
70 WHERE c.id = 1 AND c.eliminado = FALSE
71 AND NOT EXISTS (SELECT 1 FROM producto WHERE nombre = 'Calzone');
```

Anexo D — queries.sql (continuación)

```

72
73 -- HU-PROD-03: Editar producto (actualización condicional)
74 UPDATE producto
75 SET precio = COALESCE(3700.00, precio),
76     stock = COALESCE(NULL, stock)
77 WHERE id = 1 AND eliminado = FALSE;
78
79
80 -- -----
81 -- 1.3 ÉPICA: GESTIÓN DE USUARIOS
82 -- -----
83
84 -- HU-USR-01: Listar usuarios vigentes
85 SELECT id, nombre, apellido, mail, rol
86 FROM usuario
87 WHERE eliminado = FALSE
88 ORDER BY id;
89
90 -- HU-USR-02: Crear usuario (Evita duplicados en reejecuciones)
91 INSERT INTO usuario(nombre, apellido, mail, celular, contrasena)
92 SELECT 'Diego', 'Fernández', 'diego.fernandez@mail.com', '2616666666', 'hash_diego'
93 WHERE NOT EXISTS (
94     SELECT 1 FROM usuario WHERE mail = 'diego.fernandez@mail.com'
95 );
96
97 -- HU-USR-03: Baja lógica de usuario
98 UPDATE usuario
99 SET eliminado = TRUE
100 WHERE id = 5 AND eliminado = FALSE;
101
102
103 -- -----
104 -- 1.4 ÉPICA: GESTIÓN DE PEDIDOS Y TRANSACCIONES
105 -- -----
106
107 -- HU-PED-01: Consultar resumen de pedidos desde vista
108 SELECT id, usuario, fecha, estado, forma_pago, total
109 FROM v_pedidos_resumen
110 ORDER BY id;
111
112 -- HU-PED-02: Crear pedido vía Procedimiento Almacenado PL/pgSQL
113 CALL sp_crear_pedido(
114     3, -- Lucía
115     'EFECTIVO',
116     '[{"producto_id":1,"cantidad":1}, {"producto_id":10,"cantidad":2}]'::jsonb
117 );
118
119 -- HU-PED-03: Transacción manual de baja lógica de un pedido y sus detalles
120 BEGIN;
121     UPDATE detalle_pedido SET eliminado = TRUE WHERE pedido_id = 6;
122     UPDATE pedido SET eliminado = TRUE WHERE id = 6;
123 COMMIT;
124
125
126 -- =====
127 -- SECCIÓN 2: CONSULTAS ANALÍTICAS AVANZADAS Y OPTIMIZACIÓN
128 -- =====
129
130 -- -----
131 -- 2.1 AGREGACIÓN, GROUP BY Y JOINS (Top Productos y Facturación)
132 -- -----
133
134 -- Top 5 productos más vendidos por cantidad de unidades
135 SELECT pr.id, pr.nombre, SUM(dp.cantidad) AS unidades
136 FROM detalle_pedido dp
137 JOIN producto pr ON pr.id = dp.producto_id
138 WHERE dp.eliminado = FALSE
139 GROUP BY pr.id, pr.nombre
140 ORDER BY unidades DESC
141 LIMIT 5;
142
143 -- Facturación mensual por categoría
144 SELECT c.nombre AS categoria,
145     date_trunc('month', ped.fecha) AS mes,
146     SUM(dp.subtotal) AS facturado
147 FROM detalle_pedido dp

```

Anexo D — queries.sql (continuación)

```

148 JOIN pedido ped ON ped.id = dp.pedido_id AND ped.eliminado = FALSE
149 JOIN producto pr ON pr.id = dp.producto_id
150 JOIN categoria c ON c.id = pr.categoria_id
151 WHERE dp.eliminado = FALSE
152 GROUP BY c.nombre, date_trunc('month', ped.fecha)
153 ORDER BY mes, facturado DESC;
154
155 -- Productos sin ventas (LEFT JOIN + IS NULL)
156 SELECT pr.id, pr.nombre
157 FROM producto pr
158 LEFT JOIN detalle_pedido dp ON dp.producto_id = pr.id AND dp.eliminado = FALSE
159 WHERE pr.eliminado = FALSE AND dp.id IS NULL
160 ORDER BY pr.id;
161
162
163 -- -----
164 -- 2.2 RANKING DE USUARIOS POR GASTO (Funciones de Ventana / CTE / Subqueries)
165 -- -----
166
167 -- Opción A: JOIN Directo con DENSE_RANK()
168 SELECT
169     u.nombre || ' ' || u.apellido AS nombre_completo,
170     SUM(p.total) AS total_gastado,
171     DENSE_RANK() OVER (ORDER BY SUM(p.total) DESC, u.id ASC) AS puesto
172 FROM usuario u
173 INNER JOIN pedido p ON p.usuario_id = u.id AND p.eliminado = FALSE
174 WHERE u.eliminado = FALSE
175 GROUP BY u.id, u.nombre, u.apellido;
176
177 -- Opción B: CTE (Common Table Expression) con precalculado
178 WITH totales_usuarios AS (
179     SELECT
180         u.id AS usuario_id,
181         u.nombre || ' ' || u.apellido AS nombre_completo,
182         SUM(p.total) AS total_gastado
183     FROM usuario u
184     INNER JOIN pedido p ON p.usuario_id = u.id AND p.eliminado = FALSE
185     WHERE u.eliminado = FALSE
186     GROUP BY u.id, u.nombre, u.apellido
187 )
188 SELECT
189     nombre_completo,
190     total_gastado,
191     DENSE_RANK() OVER (ORDER BY total_gastado DESC, usuario_id ASC) AS puesto
192 FROM totales_usuarios;
193
194 -- Opción C: Subconsulta en FROM con HAVING
195 SELECT
196     u.nombre || ' ' || u.apellido AS nombre_completo,
197     sub.total_gastado,
198     DENSE_RANK() OVER (ORDER BY sub.total_gastado DESC, u.id ASC) AS puesto
199 FROM usuario u
200 INNER JOIN (
201     SELECT p.usuario_id, SUM(p.total) AS total_gastado
202     FROM pedido p
203     WHERE p.eliminado = FALSE
204     GROUP BY p.usuario_id
205     HAVING COUNT(p.id) >= 1
206 ) sub ON sub.usuario_id = u.id
207 WHERE u.eliminado = FALSE;
208
209 -- Verificación de equivalencia con EXCEPT (Opción A vs Opción B)
210 WITH totales_usuarios AS (
211     SELECT u.id AS usuario_id, u.nombre || ' ' || u.apellido AS nombre_completo, SUM(p.total) AS
212         total_gastado
213     FROM usuario u INNER JOIN pedido p ON p.usuario_id = u.id AND p.eliminado = FALSE WHERE u.el
214         iminado = FALSE GROUP BY u.id, u.nombre, u.apellido
215 )
216 SELECT u.nombre || ' ' || u.apellido AS nombre_completo, SUM(p.total) AS total_gastado, DENSE_RA
217     NK() OVER (ORDER BY SUM(p.total) DESC, u.id ASC) AS puesto
218 FROM usuario u INNER JOIN pedido p ON p.usuario_id = u.id AND p.eliminado = FALSE WHERE u.elimin
219     ado = FALSE GROUP BY u.id, u.nombre, u.apellido
220 EXCEPT
221 SELECT nombre_completo, total_gastado, DENSE_RANK() OVER (ORDER BY total_gastado DESC, usuario_i
222     d ASC) AS puesto FROM totales_usuarios;
223
224

```

Anexo D — queries.sql (continuación)

```
219
220 -- -----
221 -- 2.3 PRODUCTOS CON PRECIO MAYOR AL PROMEDIO DE SU CATEGORÍA
222 -- -----
223
224 -- Opción A: Subconsulta correlacionada en WHERE
225 SELECT p.id, p.nombre, p.categoria_id, p.precio
226 FROM producto p
227 WHERE p.eliminado = FALSE
228 AND p.precio > (
229     SELECT AVG(p2.precio)
230     FROM producto p2
231     WHERE p2.categoria_id = p.categoria_id
232     AND p2.eliminado = FALSE
233 );
234
235 -- Opción B: JOIN con subconsulta agrupada por categoría
236 SELECT p.id, p.nombre, p.categoria_id, p.precio
237 FROM producto p
238 INNER JOIN (
239     SELECT categoria_id, AVG(precio) AS avg_precio
240     FROM producto
241     WHERE eliminado = FALSE
242     GROUP BY categoria_id
243 ) sub ON sub.categoria_id = p.categoria_id
244 WHERE p.eliminado = FALSE
245 AND p.precio > sub.avg_precio;
246
247 -- Verificación de equivalencia con EXCEPT (Opción A vs Opción B)
248 SELECT p.id, p.nombre, p.categoria_id, p.precio
249 FROM producto p
250 WHERE p.eliminado = FALSE AND p.precio > (
251     SELECT AVG(p2.precio) FROM producto p2 WHERE p2.categoria_id = p.categoria_id AND p2.elimina
252 do = FALSE
253 )
254 EXCEPT
255 SELECT p.id, p.nombre, p.categoria_id, p.precio
256 FROM producto p
257 INNER JOIN (
258     SELECT categoria_id, AVG(precio) AS avg_precio FROM producto WHERE eliminado = FALSE GROUP B
259 Y categoria_id
260 ) sub ON sub.categoria_id = p.categoria_id
261 WHERE p.eliminado = FALSE AND p.precio > sub.avg_precio;
```

Anexo E — Script completo: transacciones.sql

Sin cambios desde el TP1. Ver §4.6 «Transacciones y concurrencia».

```

1  -- =====
2  -- Escenario 1 – Atomicidad
3  -- =====
4
5  CALL sp_crear_pedido(1, 'EFFECTIVO', '{"producto_id":1,"cantidad":999}':::jsonb);
6  SELECT * FROM pedido;
7
8  -- =====
9  -- Escenario 2 – Transacción manual (COMMIT vs ROLLBACK)
10 -- =====
11
12 BEGIN;
13 UPDATE producto SET stock = stock - 1 WHERE id = 7;
14 SELECT stock FROM producto WHERE id = 7;
15 COMMIT;
16
17 SELECT stock FROM producto WHERE id = 7;
18
19 BEGIN;
20 UPDATE producto SET stock = stock - 1 WHERE id = 7;
21 SELECT stock FROM producto WHERE id = 7;
22 ROLLBACK;
23
24 SELECT stock FROM producto WHERE id = 7;
25
26 -- =====
27 -- Escenario 3 – Aislamiento (dos sesiones al mismo tiempo)
28 -- =====
29
30 --En la seccion A
31 BEGIN;
32 SELECT stock FROM producto WHERE id = 7;
33
34 --Sin cerrar A, vamos a la seccion B
35 BEGIN;
36 UPDATE producto SET stock = stock - 5 WHERE id = 7;
37 COMMIT;
38
39 --Volvemos a la seccion A
40 SELECT stock FROM producto WHERE id = 7;
41 COMMIT;
42
43 --Repetimos lo anterior pero poniendo antes en A:
44
45 SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
46
47 BEGIN;
48 SELECT stock FROM producto WHERE id = 7;
49
50 BEGIN;
51 UPDATE producto SET stock = stock - 5 WHERE id = 7;
52 COMMIT;
53
54 SELECT stock FROM producto WHERE id = 7;
55 COMMIT;
56
57 -- =====
58 -- Escenario 4 – Bloqueo con FOR UPDATE (evitar sobreventa)
59 -- =====
60
61 SELECT id, stock FROM producto WHERE id = 3;
62
63 --En la seccion A
64 BEGIN;
65 CALL sp_crear_pedido(1, 'EFFECTIVO', '{"producto_id":3,"cantidad":8}':::jsonb);
66 --En la seccion B
67 CALL sp_crear_pedido(2, 'EFFECTIVO', '{"producto_id":3,"cantidad":8}':::jsonb);
68 --En la seccion A
69 COMMIT;
70
71

```